The VLDB Journal (2025) 34:17
https://doi.org/10.1007/s00778-024-00893-6

REGULAR PAPER

How good are multi-dimensional learned indexes? An experimental survey

Qiyu Liu[1] · Maocheng Li[2] · Yuxiang Zeng[3] · Yanyan Shen[4] · Lei Chen[2,5,6]

Received: 15 May 2024 / Revised: 30 October 2024 / Accepted: 23 December 2024 / Published online: 21 January 2025

Abstract

Efficient indexing is fundamental to managing and analyzing multi-dimensional data. A growing trend is to directly learn the storage layout of multi-dimensional data using simple machine learning models, leading to the concept of *Learned Index*. Compared to conventional indexing methods that have been used for decades (e.g., *k*d-tree and R-tree variants), learned indexes have demonstrated empirical advantages in both space and time efficiency on modern architectures. However, there is a lack of comprehensive evaluation across existing multi-dimensional learned indexes under a standardized benchmark, making it challenging to identify the most suitable index for specific data types and query patterns. This gap also hinders the widespread adoption of learned indexes in practical applications. In this paper, we present the first in-depth empirical study to answer the question: *how good are multi-dimensional learned indexes?* We evaluate **ten** recently published indexes under a unified experimental framework, which includes standardized implementations, datasets, query workloads, and evaluation metrics. We thoroughly investigate the evaluation results and discuss the findings that may provide insights for future learned index design.

Keywords Learned index · Multi-dimensional index · Benchmark

1 Introduction

Multi-dimensional data management and analytics are crucial in a wide range of fields, including business intelligence [15], smart transportation [93], neuroscience [74], climate studies [23], etc. As data volumes grow at an exponential speed, conventional multi-dimensional indexes, such as R-tree [33] and its variants [4, 9, 40, 77], have been designed to accelerate data access and query processing in large multi-dimensional databases.

Although traditional index structures like B+-tree and R-tree have been extensively studied and integrated into practical DBMSs for decades (e.g., Oracle [41] and PostgreSQL [67]), a recent proposal [44] introduced a new index design paradigm called *Learned Index* based on the observation that data indexing can be modeled as a machine learning problem, where the input is a search key, and the output is its corresponding location in storage (e.g., main memory or disk).

Consider a set of N sorted keys stored across consecutive data pages. A B+-tree index can be interpreted as a mapping

Qiyu Liu, Maocheng Li and Yuxiang Zeng have contributed equally to this work.

✉ Qiyu Liu
qyliu.cs@gmail.com

Maocheng Li
csmichael@cse.ust.hk

Yuxiang Zeng
yxzeng@buaa.edu.cn

Yanyan Shen
shenyy@sjtu.edu.cn

Lei Chen
leichen@ust.hk

1 Southwest University, Chongqing, China

2 Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong SAR, China

3 State Key Laboratory of Complex & Critical Software Environment, Beihang University, Beijing, China

4 Shanghai Jiao Tong University, Shanghai, China

5 Hong Kong University of Science and Technology (GZ), Guangzhou, China

6 HKUST Shenzhen-Hong Kong Collaborative Innovation Research Institute, Shenzhen, China

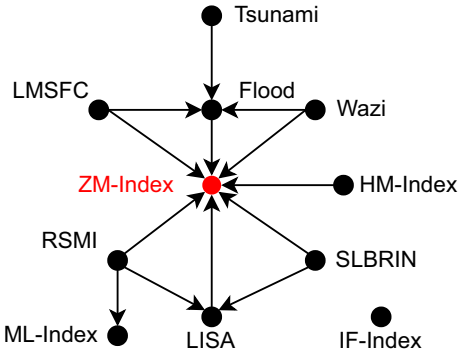


Fig. 1 Comparison results available in previous studies. An edge $A \rightarrow B$ indicates that index A is compared with index B in previous literature

from a search key x to its corresponding page ID. In this context, an error-bounded Cumulative Distribution Function (CDF) model is functionally equivalent to a B+-tree index. Since deep learning models typically rely on a heavy runtime like PyTorch [68], which is costly and less flexible, existing learned indexes prefer *stacking* simple models, such as linear functions [20], polynomial splines [44], radix splines [42], etc. Compared to conventional indexes, learned indexes are expected to be both space- and time-efficient, as a trained ML model (e.g., a piecewise linear function) is typically compact and allows for efficient inference, particularly on modern architectures.

Inspired by the impressive results obtained from 1-dimensional learned indexes [25, 44, 54, 75, 84], extensions to multi-dimensional scenarios have been intensively studied during the past years, such as ZM-Index [81], ML-Index [19], HM-Index [83], IF-Index [34], RSMI [69], LISA [47], Flood [58], Tsunami [21], LMSFC [28], Wazi [64], and SLBRIN [82]. While these studies independently claim superior empirical performance compared to conventional spatial indexes like R-tree or kd-tree variants, to the best of our knowledge, there is no comprehensive evaluation of published multi-dimensional learned indexes under a standardized experimental configuration. This lack of unified benchmarking obscures both the current impact and the future direction of this promising research area. The major limitations of existing experiments and evaluations can be summarized as follows.

First, as a rapidly evolving research field, many newly proposed indexes have not been directly compared with one another. Figure 1 visualizes the comparison relationships among existing works, where each node represents an index, and an edge $A \rightarrow B$ indicates that index A has been compared with B in previous studies. From Fig. 1, prior studies primarily compare against ZM-Index [81], which combines space-filling curves (SFC) with a 1-dimensional learned index. However, as discussed in Sect. 4.1, the original ZM-Index implementation can be further improved by leveraging recent optimizations on its underlying 1-dimensional learned index, meaning that the existing evaluation results may be outdated.

Second, existing learned indexes are not evaluated under a unified configuration, including index implementation, benchmark datasets, query workloads, and evaluation metrics. For example, a collection of multi-dimensional learned indexes [19, 21, 28, 43, 58, 64, 81, 83] internally adopt 1-D learned index (e.g., RMI [44]) as foundational components. However, different implementations of these building blocks are adopted, resulting in inconsistent comparisons and less reliable conclusions about their relative performance. Additionally, existing studies often adopt varying data models and pursue different design objectives, further complicating efforts toward a unified evaluation.

To address aforementioned limitations, we **implement** and **optimize ten** multi-dimensional learned indexes to deliver a comprehensive evaluation, which is a nearly complete coverage to the best of our knowledge. To ensure fair comparisons, we standardize experimental configurations, including index implementations (e.g., programming languages, hyper-parameter settings), benchmark datasets, query workloads, and reported metrics. Though IO efficiency and performance in distributed DBMS are also critical aspects, we limit our scope to **in-memory** and **single-machine** index performance[1], which is similar to a recent 1-D learned index benchmark SOSD [54]. Though losing generality to some extent, this empirical study is expected to deliver insightful results for in-memory analytical applications, which is of growing importance [76, 86, 90].

In summary, our experimental study makes the following technical contributions.

- To achieve fair comparisons among existing multi-dimensional learned indexes, we unify the index implementation, model training process, query workloads, and evaluation pipeline. Our benchmark implementation is made publicly available[2].
- To the best of our knowledge, our work is the first **comprehensive** and **in-depth** evaluation for multi-dimensional learned indexes. Our work also benefits future research as newly proposed learned indexes can be easily evaluated on our benchmark.
- By taking a deep dive into the evaluation results, we answer the vital question: "*How good are multi-dimensional learned indexes?*" Furthermore, we iden-

[1] Although learned indexes like LISA [47] and RSMI [69] are available for disk-based storage, they do not incorporate any disk-specific optimization objectives, such as minimizing IO requests. In practice, these indexes load the index structures into memory while storing only the data pages on disk, which is similar to other memory-based learned indexes.

[2] https://github.com/qyliu-hkust/learnedbench

tify key experimental findings that highlight promising research directions for future exploration.

The remainder of this paper is structured as follows. In Sect. 2, we formulate the multi-dimensional data model and overview the background of multi-dimensional indexes. Section 3 establishes a taxonomy and surveys existing multi-dimensional learned indexes. Section 4 introduces index implementation details and benchmark setups. In Sect. 5, we present and investigate evaluation results. Finally, Sect. 6 concludes the paper and discusses directions for future research.

2 Preliminaries and background

In this section, we provide an overview of data models, query definitions, and both conventional and learned indexing methods in multi-dimensional space.

2.1 Multi-dimensional data model

As adopted by most of the existing works, we consider a collection of N points or boxes from d-dimensional Euclidean space, and we focus on range queries and k nearest neighbor queries (kNN) defined as follows.

Definition 1 (Point) A point $\mathbf{x} \in \mathbb{R}^d$ is a vector in d-dimensional Euclidean space. Let $\mathbf{x}^i$ denote the i-th coordinate of $\mathbf{x}$. For two points $\mathbf{x}_1, \mathbf{x}_2$, we define $\mathbf{x}_1 \preceq \mathbf{x}_2$ if $\mathbf{x}_1^i \leq \mathbf{x}_2^i$ for $\forall i = 1, \cdots, d$.

Definition 2 (Bounding Box) A box is defined as a d-dimensional hyper-rectangle $R = (\mathbf{x}_\ell, \mathbf{x}_h)$, where $\mathbf{x}_\ell, \mathbf{x}_h \in \mathbb{R}^d$ denote the bottom-left and top-right corner points of R, respectively. A box R degenerates to a point when $\mathbf{x}_\ell = \mathbf{x}_h$ (i.e., $\mathsf{area}(R) = 0$).

Definition 3 (Range Query) Given a collection of geometries $\mathcal{G}$ and a query box $R^{\mathsf{Q}} = (\mathbf{x}_\ell^{\mathsf{Q}}, \mathbf{x}_h^{\mathsf{Q}})$, a range query retrieves all geometries (points or boxes) covered by R^{Q}, i.e., $\mathsf{range}(R^{\mathsf{Q}}) = \{g \mid g \in \mathcal{G} \wedge R^{\mathsf{Q}} \,\mathsf{covers}\, g\}$. If g is a point $\mathbf{x}$, $R^{\mathsf{Q}} \,\mathsf{covers}\, g = \mathbf{x}_\ell^{\mathsf{Q}} \preceq \mathbf{x} \preceq \mathbf{x}_h^{\mathsf{Q}}$; and if g is a box $R = (\mathbf{x}_\ell, \mathbf{x}_h)$, $R^{\mathsf{Q}} \,\mathsf{covers}\, g = \mathbf{x}_\ell^{\mathsf{Q}} \preceq \mathbf{x}_\ell \wedge \mathbf{x}_h \preceq \mathbf{x}_h^{\mathsf{Q}}$.

Definition 4 (k-Nearest Neighbor Query) Given a distance metric between two geometries $dist(\cdot, \cdot)$, a k-nearest neighbor query $\mathsf{kNN}(q, k)$ retrieves k geometries from $\mathcal{G}$ whose distances to q are ranked in ascending order. Formally, for $\forall g \in \mathsf{kNN}(q, k), \nexists g' \in \mathcal{G} \backslash \mathsf{kNN}(q, k)$ such that $dist(g', q) \leq dist(g, q)$.

Notably, *all* of the multi-dimensional learned indexes discussed and evaluated in this work adopt point geometry as the data model. However, extending indexes designed for points to support boxes, which can be regarded as a generalization to points, is straightforward, as each box can be represented by two corner points, $\mathbf{x}_\ell$ and $\mathbf{x}_h$. Queries involving boxes (e.g., range queries) can be processed by first retrieving candidate boxes using the index, then refining the results by applying spatial predicates such as covers, touch, and intersects [12].

2.2 Conventional data indexing

Multi-dimensional indexes and related query processing techniques have been extensively studied by academia and deployed in commercial DBMS for decades.

For low-dimensional data, conventional indexes can be classified into three classes based on different space partition strategies: R-tree variants, kd-tree variants, and grids. ❶ R-trees [33] organize data using minimum bounding rectangles (MBRs) to encapsulate spatial objects, where each internal node represents a MBR covering its child nodes. Variants of the R-tree, such as R*-tree [9], STR-tree [46] and Hilbert R-tree [40], focus on minimizing overlap between nodes by different *packing* strategies, improving query performance. ❷ kd-trees [10] are binary trees that recursively partition data space into axis-aligned hyperplanes. Compared to R-trees, kd-trees alternately split along different dimensions at each level, creating a structure well-suited to kNN queries in multi-dimensional space. Variants like kd-B-tree [72] extend kd-tree by adding paging capabilities, making it more suitable to disk-based storage. ❸ Grids [59] are simple yet effective spatial indexing structures that partition the data space into fixed cells, ideal for dense data distributions. Adaptive variants like quad-tree [26] refine grid cells further in dense areas, thus improving spatial locality and query efficiency.

In high-dimensional spaces, however, data sparsity inevitably increases due to the curse of dimensionality, making kNN processing based on the above indexes often perform no better than a naïve linear scan approach. To this end, pivot-based methods are commonly adopted to index data in high-dimensional space (or more generic metric space), including iDistance [38], vantage-point (VP) tree [89], MVP-tree [14], etc. A recent survey [16] summarized and evaluated pivot-based indexes for high-dimensional query processing. Due to the inherently high cost of query processing in high-dimensional space, exact query methods are hard to scale to billion-scale high-dimensional vector sets. To handle web-scale vectors, approximate nearest neighbor (ANN) indexes have been intensively studied, falling into three major categories: locality-sensitive hashing (LSH)-based [2, 18, 48], proximity graph-based [52, 53], and product quantization (PQ)-based [29, 39, 87] approaches. A recent experimental study [7] provided a detailed benchmark for ANN indexes.

2.3 Learned data indexing

1-Dimensional Learned Index. A 1-D learned index intrinsically functions as an error-bounded CDF model, scaled by the data size N. In their seminal work [44], Kraska et al. proposed the first learned index, RMI, which has been empirically shown to be Pareto optimal when compared to a B+-tree index. Following RMI, Kip et al. proposed a simpler learned index RadixSpline [42], which requires only a single pass of data for construction. Furthermore, PGM-Index [25] adopted the optimal piece-wise linear approximation [62] as the underlying CDF model, yielding strong theoretical results regarding space and time complexity. To accommodate dynamic operations such as key insertion and deletion, Ding et al. proposed ALEX [20] by using a gapped array structure to handle record updates. LIPP [84] further enhanced update efficiency by minimizing the last-mile search error in leaf nodes.

More detailed discussions and comparisons of 1-D learned indexes can be found in a recent benchmark paper SOSD [54]. In addition to data indexing, there are also emerging efforts to integrate learned models into conventional data structures and algorithm designs, such as learned Bloom filters [44, 51, 55], learned sorting [45], learned data compression [11, 50], etc.

Multi-Dimensional Learned Index. Designing multi-dimensional learned indexes has recently emerged as a promising research direction. Similar to how an R-tree is a multi-dimensional analog to a B+-tree index, it is natural to extend 1-dimensional learned indexes to multi-dimensional space by directly learning the mapping from multi-dimensional keys to their storage location. Typical works in this area include ZM-Index [81], ML-Index [19], HM-Index [83], IF-Index [34], RSMI [69], LISA [47], Flood [58], Tsunami [21], LMSFC [28], Wazi [64], and SLBRIN [82]. Additionally, a recent system SageDB [43] also incorporates a learned grid index, which can be viewed as a simplified version of Flood [58]. The specific details of these learned indexes will be discussed in Sect. 3.

Notably, unlike conventional indexes and even 1-D learned indexes, current multi-dimensional learned indexes primarily demonstrate performance superiority through empirical evidence, lacking sufficient theoretical support. Recent advances [24, 49] claim that, on N 1-D sorted keys, learned indexes can process lookup queries in sub-logarithmic time $O(\log\log N)$ using linear space $O(N)$, which is *theoretically better* than a B+-tree. It would be an interesting research direction to extend these results from 1-dimensional to multi-dimensional scenarios.

3 Multi-dimensional learned index

We investigate data layouts adopted by different learned indexes in Sect. 3.1. Then, we develop an index taxonomy in Sect. 3.2 and overview indexes in each category in Sect. 3.3–3.5. Table 1 highlights key technical features of existing multi-dimensional learned indexes.

3.1 Data layout

Data layout specifies how an index organizes multi-dimensional data in storage (i.e., disk or memory), forming the basis of subsequent learned index taxonomy. As shown in Table 1, existing learned indexes typically employ either an ❶ order-based layout or a ❷ space partition-based layout.

Order-based Layout. Multi-dimensional data are organized in storage according to a pre-defined sorting order. Unlike the 1-D case, multi-dimensional data lacks an intrinsic sorting order. Thus, existing works typically select a sorting dimension or employ the space-filling curves (SFC), such as Z-order curve [70] and Hilbert curve [40]), to order the data.

Partition-based Layout. The partition-based layout recursively divides the data space under a specific strategy (e.g., the middle-point strategy in kd-tree [10]) until reaching a partition threshold (e.g., data size below a pre-specified block size). Data in each partition are then grouped and sequentially stored. A grid-based layout is a special case of the partition-based layout, where the data space is divided into grid cells, and points in the same cell are stored consecutively.

3.2 Taxonomy

Based on different data layouts and how ML models are integrated into the index structure, we classify existing multi-dimensional learned index studies into three categories: ❶ projection-based index, ❷ augmentation-based index, and ❸ grid-based index.

Projection-based Index adopts a mapping function $f : \mathbb{R}^d \mapsto \mathbb{R}$ to project d-dimensional keys to 1-D values and then trains a 1-D learned index (e.g., RMI [44] or PGM-Index [25]) on the mapped values. To preserve spatial locality, existing studies usually employ space-filling curves like Z-order or Hilbert curves as the projection function. Such an idea is not new and has been widely studied in conventional spatial index design (e.g., UB-tree [71] and Hilbert R-tree [40]). The projection-based index design is the most popular approach in existing multi-dimensional learned indexes, including LISA [47], ZM-Index [81], ML-Index [19], HM-Index [83], LMSFC [28], Wazi [64], and SLBRIN [82].

Augmentation-based Index extends traditional index structures, which are based on recursive space partitioning, but incorporates ML model-based search in place of standard

Table 1 Key technical features of the existing multi-dimensional learned index structures

Index	Type	Data ordering	Data layout	Model	Model training	Workload-aware	Update	Dim.	Supported queries
ZM-Index [81]	projection	Z-curve	order-based	RMI	algorithmic	×	×	≥ 2	Range
Wazi [64]	projection	Z-curve	order-based	RMI	algorithmic	✓	×	≥ 2	Range
HM-Index [83]	projection	Hilbert-curve	order-based	RMI	algorithmic	×	×	≥ 2	Range
ML-Index [19]	projection	projection function	order-based	RMI	algorithmic	×	×	≥ 2	Range, *k*NN
LISA [47]	projection	projection function	order-based	piecewise linear	numpy	×	✓	≥ 2	Range, *k*NN
SLBRIN [82]	projection	GeoHash	order-based	MLP	TensorFlow	×	×	≥ 2	Range
LMSFC [28]	projection	learned SFC	order-based	piecewise linear	algorithmic	✓	✓	≥ 2	Range
IF-Index [34]	augmentation	selected dimension	partition-based	linear	algorithmic	×	✓	≥ 2	Range
RSMI [69]	augmentation	Z-curve	partition-based	MLP	PyTorch	×	✓	2	Range, *k*NN, ANN
Flood [58]	grid	selected dimension	partition-based	RMI	algorithmic	✓	×	≥ 2	Range
Tsunami [21]	grid	selected dimension	partition-based	RMI	algorithmic	✓	×	≥ 2	Range

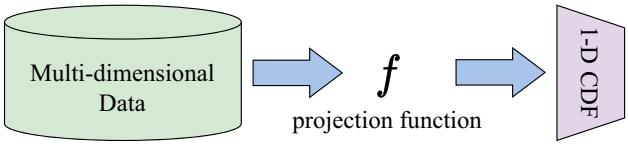


Fig. 2 Workflow of the projection-based index

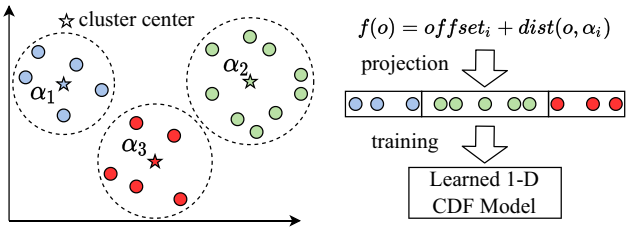


Fig. 3 The projection function of ML-Index [19] where the k-means centers are used as reference points

node traversal. These indexes employ space partitioning and node splitting strategies similar to conventional multi-dimensional indexes like R-tree and kd-tree, thus inheriting their high generality. This class of indexes include IF-Index [34] and RSMI [69].

Grid-based Index employs grids as the data layout. Unlike typical grid indexes, learned grid indexes do not physically store grid cells (i.e., partition boundaries); instead, they train learned CDF functions over selected dimensions to locate the appropriate grid cell. For example, suppose the j-th dimension in the grid index is partitioned into m buckets, a point $\mathbf{x}$ will be placed into the $\lfloor \mathsf{cdf}^j(\mathbf{x}^j) \cdot m \rfloor$-th bucket, where $\mathsf{cdf}^j(\cdot) \in [0, 1]$ is the CDF function on the j-th dimension of all points. The grid-based indexes include Flood [58] and Tsunami [21].

Additionally, recent studies, such as Qd-tree [88], RLR-tree [32] and AI+R-tree [35], also leverage deep reinforcement learning (DRL) to assist in constructing multi-dimensional data index. These DRL-based methodologies focus on optimizing the structures and data layouts of conventional indexes like R-tree. In these works, DRL models are used to find a better data layout rather than to locate multi-dimensional keys in storage. Therefore, they are out of the scope of "learned index" and thus will not be evaluated in our work.

3.3 Projection-based index

Figure 2 illustrates the workflow of projection-based indexes. For a d-dimensional dataset $\mathcal{X}$, a projection function $f : \mathbb{R}^d \to \mathbb{R}$ is applied to project $\mathcal{X}$ to a set of 1-D values $\mathcal{X}' = \{f(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}\}$. Then, $\mathcal{X}'$ is sorted and sequentially stored such that a 1-D learned index (e.g., RMI or PGMIndex) can be constructed to fit the mapping from projected values to corresponding storage locations. To ensure the correctness of range query processing, the projection function f should be a monotonic mapping. For two points $\mathbf{x}$ and $\mathbf{x}'$, $f(\mathbf{x}) \le f(\mathbf{x}')$ holds if $\mathbf{x}'$ dominates $\mathbf{x}$ on each dimension, i.e., $\mathbf{x} \preceq \mathbf{x}'$. Then, any range search query can be transformed to 1-D interval search over the mapped values. The major difference of existing works in this class is the choice of projection function f.

ZM-Index [81] is the *first* multi-dimensional learned index where the Z-order curve is chosen as the projection function. ZM-Index partitions the space into grids and adopts the bit interleaving trick to efficiently compute the Z-address [71]. For range queries, the query box is transformed to intervals of Z-addresses, such that the underlying 1-D learned index (e.g., an RMI) can be queried to efficiently locate the query box in storage. **HM-Index** [83] and **SLBRIN** [82] are two similar projection-based indexes that use the Hilbert-curve and Geo-Hash, respectively, as their projection functions. **Wazi** [64] and **LMSFC** [28] are two improved versions of the original ZM-Index by adaptively optimizing the Z-order curve using query workloads.

ML-Index [19] adopts an improved iDistance function as the projection function, which is typically used to index high dimensional data [38]. As shown in Fig. 3, given a set of m reference points (RP) $\mathbf{r}_1, \cdots, \mathbf{r}_m$, which is usually obtained by k-means, the input data $\mathcal{X}$ are partitioned into m disjoint subsets based on the distance to each RP. For any $\mathbf{x} \in \mathcal{X}$, supposing that $\mathbf{r}_i$ is the closest RP to $\mathbf{x}$, the ML-Index adopts the following projection function,

$$f(\mathbf{x}) = dist(\mathbf{x}, \mathbf{r}_i) + \sum_{j<i} \max_{\mathbf{x}' \in \mathcal{X}_j} dist(\mathbf{x}', \mathbf{r}_j), \tag{1}$$

where $\mathcal{X}_j = \{\mathbf{x} \mid \mathbf{r}_j$ is the closest RP to $\mathbf{x} \wedge \mathbf{x} \in \mathcal{X}\}$.

Recall the iDistance computation, i.e., $iDist(\mathbf{x}) = dist(\mathbf{x}, \mathbf{r}_i) + i \cdot C$ (with a large constant C) and $\mathbf{r}_i$ is the closest RP to $\mathbf{x}$. Compared to the original iDistance method, Eq. (1) removes overlaps between different partitions and reduces gaps between consecutive partitions, making the projected 1-D values easier to learn by a CDF model. The query processing (range queries or kNN queries) using ML-Index is similar to the iDistance [38] method. Since evaluating Eq. (1) only requires a valid distance metric $dist(\cdot, \cdot)$, ML-Index also supports data in general metric spaces (e.g., strings and graphs). However, we focus on the performance of ML-Index in Euclidean space to compare with other multi-dimensional learned indexes.

LISA [47] addresses inefficiencies in SFC-based projection, which often accesses irrelevant data blocks. Instead, LISA applies a grid-based projection function. For a d-dimensional dataset, data points are partitioned into $T_1 \times T_2 \cdots \times T_d$ *equal-depth* grid cells, and each cell C is assigned a unique ID t. Let $C_t = [\theta_l^{(1)}, \theta_h^{(1)}) \times \cdots \times [\theta_l^{(d)}, \theta_h^{(d)})$, where $\theta_l^{(j)}, \theta_h^{(j)}$

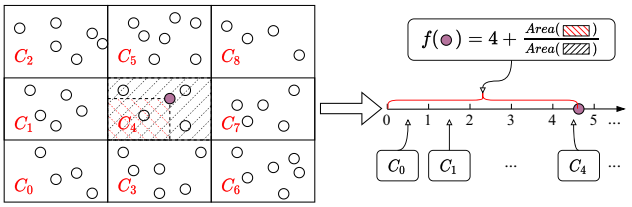


Fig. 4 The projection function of LISA [47] based on a 3 × 3 grid partition. Notably, the Lebesgue measure in 2-D space is the area of a rectangular region

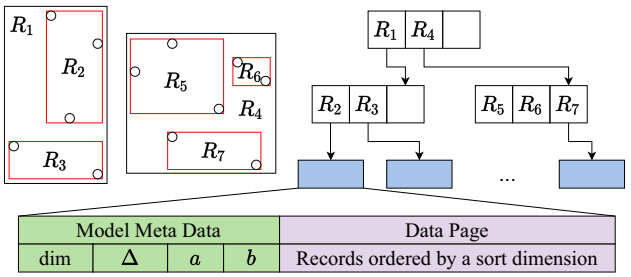


Fig. 5 An IF-Index [34] where dim is the selected sorting dimension, Δ is the maximum prediction error, and a, b are the slope and interception of the linear model

are grid boundaries along the j-th dimension. Then, for any point $\mathbf{x}$ falling into C_t, LISA's projection function is defined as follows,

$$f(\mathbf{x}) = t + \frac{\lambda(H_t)}{\lambda(C_t)}, H_t = [\theta_l^{(1)}, \mathbf{x}^1) \times \cdots \times [\theta_l^{(d)}, \mathbf{x}^d), \quad (2)$$

where $\lambda(\cdot)$ is the Lebesgue measure. As shown in Fig. 4, the areas of red and black dashed regions are the Lebesgue measures (2D) of C_4 and H_4 for the point highlighted in purple. According to Eq. (2), points falling into cell C_t will be mapped to the same interval $[t, t+1)$, thus preserving spatial locality.

After constructing the grid, all data points are projected to 1-D space using Eq. (2) and partitioned to shards. A *shard prediction function* is then trained to map each point to its shard ID. Finally, points belonging to the same shard are stored in data pages, and a local model (i.e., 1-D learned index) is trained to locate the correct data page. Notably, although LISA requires a grid partition, it does not store a physical grid index. Instead, LISA encodes grid information into the projection function (i.e., Eq. (2)) and orders data by the projected values. Thus, we classify LISA as a projection-based index rather than a grid-based index.

3.4 Augmentation-based index

We then introduce the augmentation-based indexes, where learned models are plugged into conventional index structures (e.g., R-tree or kd-tree) to accelerate the internal node search efficiency.

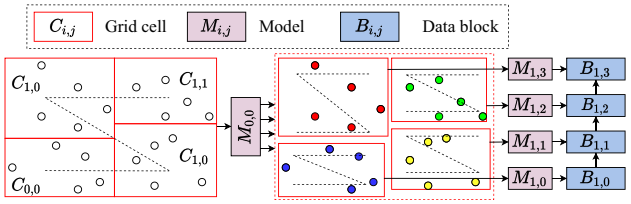


Fig. 6 Illustration of RSMI [69] where the partition threshold $N' = 5$. Note that, each data block maintains a pointer to the next block for efficient scanning

IF-Index [19] replaces leaf node search in conventional tree-based structures, e.g., R-tree, with a model-based search strategy. The IF-Index structure based on an R-tree is illustrated in Fig. 5. The non-leaf nodes are constructed and stored following a standard R-tree index, while leaf nodes keep both data pages and extra model metadata. The model metadata includes a sorting dimension dim and a linear interpolation model to predict the search key's location in a data page. For each leaf node, IF-Index determines the best sorting dimension by minimizing the interpolation cost.

RSMI [69] takes a further step by employing model-based searches in both leaf and non-leaf nodes. As shown in Fig. 6, to construct the RSMI index, spatial points (2D) are partitioned into $2^{\lfloor \log_4 N'/B \rfloor} \times 2^{\lfloor \log_4 N'/B \rfloor}$ *equal-depth* grid cells, where B is the block size and N' is the partition threshold such that a learned function can achieve high accuracy. Grid cells are then sorted by Z-order curves, and a model is trained to map each grid cell to its sorting index (e.g., $M_{0,0}$ in Fig. 6). As each grid cell can still contain many points (i.e., $\gg N'$), the above partitioning and model training procedures are recursively invoked until each partition has at most N' data points. Data points are then packed into blocks by Z-order rank, with leaf models trained to predict each point's block ID (e.g., $M_{1,0} \sim M_{1,3}$ in Fig. 6).

Similar to a quad-tree, RSMI recursively partitions space using a grid tree structure. The learned models in RSMI are used to perform efficient searches when traversing the grid tree, instead of directly locating grid cells. Therefore, RSMI is regarded as an augmentation-based index rather than a grid-based index.

3.5 Grid-based index

The grid-based indexes, Flood [58] and Tsunami [21], aim to learn compact multi-dimensional grids to efficiently process range predicates.

Flood [58] adopts the common grid index as the data layout. To construct Flood on a d-dimensional dataset, a sorting dimension is chosen to order data within each grid cell, while the other $d-1$ dimensions are adopted to overlay

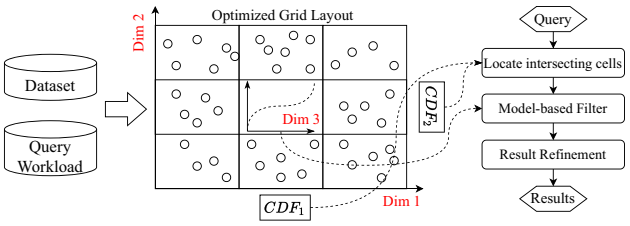


Fig. 7 Flood [58] framework on a 3D dataset, where Dim1 and Dim2 are used to generate a grid layout, and Dim3 is used to sort the data within each grid cell

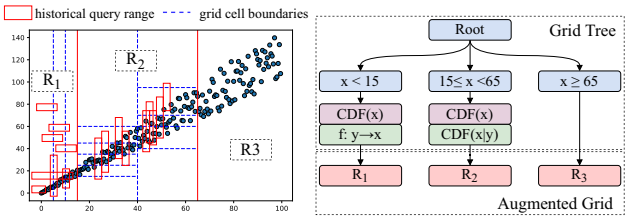


Fig. 8 Illustration of a Tsunami [21] index built on a 2-dimensional correlated dataset and tuned by skewed query workloads. The red solid lines refer to the partition boundaries of the grid tree, and the blue dashed lines refer to the bucket boundaries of augmented grids

a grid. W.l.o.g., we assume the d-th dimension serves as the sorting axis. Different from conventional grid index, Flood employs learned CDF models (i.e., 1-D learned index) to partition grids. Flood organizes the underlying grids as a multi-dimensional array G of cells. For any point $\mathbf{x}$, its corresponding cell in G is given by:

$$G(\mathbf{x}) = (\left\lfloor \mathsf{cdf}_1(\mathbf{x}^1) \cdot K_1 \right\rfloor, \left\lfloor \mathsf{cdf}_2(\mathbf{x}^2) \cdot K_2 \right\rfloor, \cdots, \left\lfloor \mathsf{cdf}_{d-1}(\mathbf{x}^{d-1}) \cdot K_{d-1} \right\rfloor), \quad (3)$$

where $\mathsf{cdf}_i(\cdot)$ is the CDF trained for the i-th dimension, and K_i is the partition number for dimension i. To enable faster refinement of a range filter, each grid cell includes an additional CDF model trained on the sorting dimension.

As shown in Fig. 7, Flood adopts historical query workloads to pick the sorting dimension and tune hyper-parameters (e.g., K_i values). To process a range query, cells intersecting the query box are first retrieved via Eq. (3), and then intra-cell CDF models are queried to provide efficient filtering based on the range predicates.

Tsunami [21] further improves Flood's performance on correlated data and skewed query workloads. As shown in Fig. 8, Tsunami comprises two components: a grid tree that adapts to skewed query workloads and an augmented grid index optimized for capturing data correlations. The grid tree, similar to a kd-tree index, partitions the space along selected dimensions, creating disjoint regions that minimize query skewness within each partition (e.g., $R_1 \sim R_3$ in Fig. 8). Each region is then augmented with a grid structure as follows.

Unlike Flood, where grid cells are independently determined by CDF models on each dimension, Tsunami captures correlations via functional dependencies (e.g., $f : y \rightarrow x$ in R_1) and conditional CDF models (e.g., $\mathsf{cdf}(x|y)$ in R_2). Grid granularity is tuned based on historical workloads, with frequently queried areas being intensively partitioned (e.g., R_2) while less-frequent areas being more coarsely partitioned (e.g., R_1 and R_3). Range queries in Tsunami are processed similarly to those in Flood, by locating the intersected regions and grids and then refining the results. Additionally, both Flood and Tsunami include a periodical mechanism to monitor workload shifts, enabling them to recalibrate the grid layout and models in response to significant shifts in query distribution.

3.6 Discussion

In this section, we discuss additional design considerations for learned indexes, including: ❶ data model, ❷ query support, ❸ dynamic updates, and ❹ ML model selection and training.

Data Model. Existing multi-dimensional learned indexes typically adopt the point data model as defined in Sect. 2.1. However, since bounding boxes are also an important and generic data model in various spatial applications, this benchmark study implements support for box geometries by treating each bounding box as two corner points. Queries over boxes will thus be processed as point queries, followed by an additional refinement step to ensure accurate results.

Query Support. The indexes included in this benchmark primarily focus on exact range and kNN queries in low- to medium-dimensional spaces ($d < 10$), covering a wide spectrum of applications, such as spatial/RDBMS filtering, location recommendation, and image processing [17]. As shown in Table 1, all benchmark indexes support range queries, while only ML-Index, LISA, SLBRIN, and RSMI explicitly support kNN queries. Although kNN queries can be implemented using range query as a primitive, this straightforward approach is typically less efficient. Thus, in subsequent experiments, we will evaluate kNN queries only for the indexes with specific optimizations.

Index Update. Most multi-dimensional learned indexes do not support dynamic operations like insertion and deletion, due to the difficulties in updating outdated models. An interesting finding is that, mutable learned indexes like RSMI [69] and LISA [47] all adopt a model-based layout to handle dynamic updates. To insert/delete a record, RSMI and LISA first query the underlying learned models to obtain the appropriate block ID, and insertion proceeds if the corresponding block is not full. In this case, the predicted block ID is always considered to be *correct* due to the model-based layout.

This strategy mirrors that of existing updatable 1-D learned indexes (e.g., ALEX [20] and LIPP [84]), where new

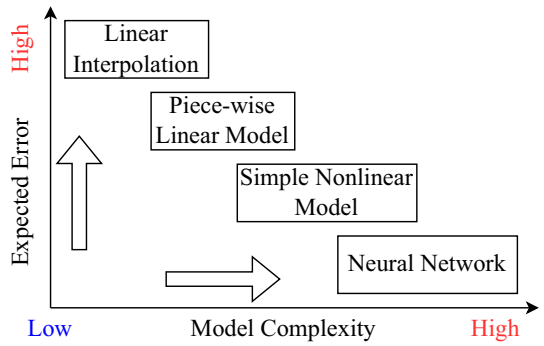


Fig. 9 Characteristics of different model choices

keys are inserted to a sorted array with gaps based on the model predictions. However, existing indexes merely consider the query performance decay problem when the data distribution significantly shifts, and in the worst case, reconstructing the index is inevitable.

Model Selection and Training. A learned index can be viewed as a combination of "data layout + learned model". The choice of the underlying model significantly impacts index performance. As depicted in Fig. 9, simple models, such as linear interpolation or piece-wise linear approximation (PLA), offer efficient training and inference but may lead to higher error rates due to limited capacity. Conversely, complex models, like neural networks, can achieve greater accuracy but come with increased training and inference overheads.

Effective model training is also crucial. Existing approaches either use custom algorithms or rely on mature libraries like Pytorch [68], Tensorflow [78], or numpy [60]. While these libraries provide flexibility in model design, they usually introduce additional runtime overhead and longer training time. In contrast, well-optimized algorithmic methods, like the error-bounded piece-wise linear approximation [62] in PGM-Index [25] or the top-down training strategy in RMI [44], are generally more efficient. Our evaluation shows that while library-based models may offer slight improvements in accuracy, they require significantly more time to train-up to $\sim 9000\times$ longer than optimized algorithmic solutions.

4 Experiment setup

This section outlines index implementation details and experimental setups. The entire benchmark is implemented in C++ and compiled by g++ 9.5 with optimization level of O3. All experiments are conducted on a Ubuntu desktop equipped with an Intel Core™ i7-10700K CPU and 32 GB of DDR4 RAM. To ensure consistent performance, we disable the CPU's Turbo Boost feature and lock the frequency at 4.67 GHz.

4.1 Index implementation details

We first present the implementation details of compared multi-dimensional indexes. Table 2 summarizes all 17 compared indexes (10 learned and 7 non-learned).

❶ **`ZMI`**. To compute Z-order values (a.k.a., Z-addresses), for a dataset of N d-dimensional points, we uniformly partition each dimension into $N^{1/d}$ buckets, implying an equal-width grid of N buckets. While a finer grid partition enhances the pruning power by reducing unnecessary data access, it also increases the space and time costs of index construction and query processing. We try different grid partition resolutions and find that a uniform grid of $N^{1/d} \times \cdots \times N^{1/d}$ consistently achieves the best performance on various datasets.

❷ **`HMI`** and ❸ `SLBRIN`. These two indexes can be viewed as `ZMI` with different projection functions. We adopt a similar grid partition method to compute Hilbert-curve value and GeoHash for `HMI` and `SLBRIN`, respectively.

❹ **`Wazi`** and ❺ `LMSFC`. These two indexes are adaptive versions of `ZMI`, and the computation of Z-addresses is based on non-uniform grid partitions, which can be modeled and tuned by sample data and queries. As the common open datasets do not include their query workloads, we follow the instructions in [28] to generate 1000 *uniform* queries to construct the index.

❻ **`MLI`**. We determine the reference points by invoking Lloyd's k-means algorithm with the k-means++ initialization strategy [5]. As noted in [38], the efficiency gains from additional reference points diminish beyond 25 for pivot-based indexes. In our implementation, the partition number P is set to 20 for datasets of size $<$ 20M and 40 for datasets of size $>$ 20M.

❼ **`LISA`**. The original `LISA` implementation is in Python and highly relies on Numpy. To maintain consistency with our C++-based benchmark, we re-implement and optimize the LISA index structure. To compute Eq. (2), we construct an equal-depth grid where each grid cell contains roughly $B = 2000$ points. By such setting, the range of Eq. (2) is in $[0, N/B + 1]$.

❽ **`IFI`**. We adopt an IF-Index built on R-tree. The original IF-Index employs linear interpolation to estimate key locations, which can lead to significant prediction errors, especially with non-uniform data. Instead, our implementation trains linear models using the least square method, resulting in a 15% performance boost at a cost of 20% more training time. To ensure that the index can achieve the best performance, we enumerate all possible sorting dimensions and choose the one that minimizes search cost over a given probe query set.

❾ **`RSMI`**. We select their original implementation and follow the same index tuning strategies as discussed in [69].

Table 2 Summary of the compared indexes. *RSMI and ANN support both exact and approximate *k*NN search

Index	Range	*k*NN	Updatable	Require Workload
ZMI	✓	✓	×	×
HMI	✓	✓	×	×
SLBRIN	✓	✓	×	×
Wazi	✓	✓	✓	✓
LMSFC	✓	×	✓	✓
MLI	✓	✓	×	×
RSMI	✓	✓*	✓	×
IFI	✓	×	✓	×
LISA	✓	✓	✓	×
Flood	✓	×	×	✓
STRtree	✓	✓	✓	×
R*tree	✓	✓	✓	×
kdtree	✓	✓	✓	×
octree	✓	✓	✓	×
ANN	×	✓*	✓	×
UG	✓	×	✓	×
EDG	✓	×	✓	×

Notably, RSMI is a pure spatial index that only supports 2-D datasets. The model training and inference of RSMI are based on Pytorch [68]. To ensure a fair comparison with other indexes, we use the CPU-only version of Pytorch while enabling multi-threading to accelerate model training (otherwise it fails to terminate in 5 hours for a 20M dataset).

❿ **Flood**. As Flood is not open-sourced currently, we implement it at our best. To prevent the number of grid cells from growing exponentially with the data dimension, we set the partition number for each dimension (except the sorting dimension) to $(N/B)^{1/(d-1)}$, where $B = 2000$ is the expected number of points in each cell. The sorting dimension within each cell is selected similarly to IFI. We do not include Tsunami [21] in the benchmark, as it can be regarded as a workload-driven Flood index and most of the compared indexes are not specifically optimized using query histories, thus creating an unfair comparison.

Except RSMI and IFI, which employ deep learning and linear regression models, we select the PGMIndex [25] as the default underlying 1-D learned index for the multi-dimensional learned indexes to ensure consistency in their implementations. The considerations for not using RMI [44], another popular learned 1-D index, for our benchmark implementation is given as follows.

- The state-of-the-art RMI implementation [54] functions as an "index compiler", taking a dataset as input and generating *static* index data and header files separately, which is less flexible to be integrated with other index structures.
- According to a recent theoretical revisit of 1-D learned index [49], PGM-Index achieves a sub-logarithmic query time and is empirically as efficient as RMI by adopting an optimized search operator.
- Parameter tuning of PGM-Index is much easier than RMI as only an error parameter ϵ is required, while RMI requires tuning of the error parameters, number of models, and model types for each level of the index. We tune the underlying PGM-Index for each index by using the cost model introduced in [49].
- In this experimental study, we focus on the effects of different index design choices (e.g., data layout and projection function) within a unified environment, rather than on replaceable components that have a minor impact on the *relative* performance.

In addition to learned indexes, we also implement and compare 8 non-learned baselines that are commonly used in practice.

❶ FS: the naïve full scan algorithm. To speed up query processing, we sort the points based on Z-addresses to allow the pruning of unnecessary points.

❷ R*tree: the R*-tree index [9] that optimizes the R-tree structure by minimizing the leaf node overlap.

❸ STRTree: the R-tree index with the sort-tile-recursive (STR) bulk-loading strategy [46]. For both R*tree and STRTree, we adopt the implementations from the Boost Geometry Library [12]. The R-tree node capacity is set to 128 according to a recent benchmark [66].

❹ kdtree: the *k*d-tree index [10] that recursively partitions the space by the median of each dimension in a round-robin fashion. We adopt the implementation in the nanoflann project [57].

❺ ANN: a *k*d-tree-based library [3] providing both exact and approximate *k*NN search. If a non-negative error parameter ϵ is provided, ANN ensures the retrieved i-th nearest neighbor is a $(1+\epsilon)$-approximation to the true i-th nearest neighbor by using the priority search algorithm [6]. In our benchmark, we tune ϵ in range $[0, 5]$ to control the trade-off between query efficiency and result accuracy.

❻ octree: the generalized octree index that can be regarded as an adaptive and recursive grid index. We adopt the implementation from GEOS library [30]. For 2-D data, octree is also known as the quad-tree index.

❼ UG: a uniform grid (a.k.a., equal-width grid). The partition number of each dimension is set to $\sqrt[d]{N/2000}$.

❽ EDG: an equal-depth grid index where each dimension is sorted and partitioned into $\sqrt[d]{N/2000}$ parts of equal size. Compared to a uniform grid, EDG better accommodates skewed data distributions at the cost of higher construction complexity.

Table 3 Summary of datasets.

Dataset	Type	Size	Spatial Dim.	Total Dim.
FourSquare	Spatial	3.7 M	2	2
OSM	Spatial	63 M	2	2
Toronto3d	Mixed	21 M	3	7
Taxi	Mixed	47 M	4	9
Tpch	Relational	100 M	0	8
Stock	Relational	21 M	0	7
Uniform	Synthetic	5–100 M	2–8	2–8
Normal	Synthetic	5–100 M	2–8	2–8
Lognormal	Synthetic	5–100 M	2–8	2–8

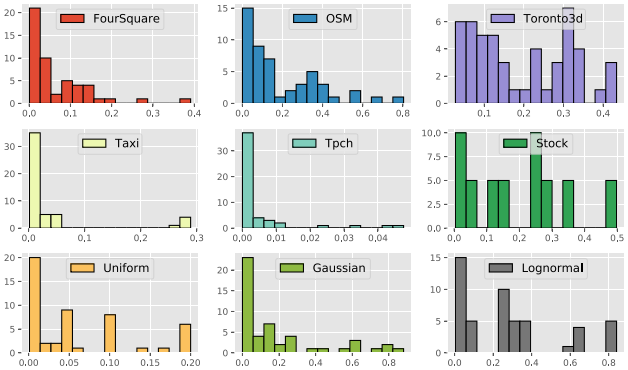


Fig. 10 Selectivity distribution of generated range queries. The x-axis and y-axis refer to the query selectivity and number of queries, respectively

4.2 Datasets and query generation

We evaluate all the compared methods in Sect. 4.1 on both real and synthetic datasets, covering a wide range of data sizes, dimensionalities, and distribution skewness. Table 3 summarizes the dataset statistics.

Real Datasets. We adopt six commonly used datasets from real world applications.

❶ `FourSquare` is a location dataset extracted from a geo-social network [27]. Each record represents the coordinates of a co-location event involving two users in the social network.

❷ `OSM` is a recent dump of OpenStreatMap [63]. We focus solely on point objects (i.e., pairs of latitude and longitude), excluding complicated geometries like polygons and line-strings as querying these geometries is not commonly supported by existing learned indexes (see discussions in Sect. 2.1).

❸ `Toronto3d` is a dataset of LiDAR images of urban roadways [79], containing spatial coordinates (i.e., x, y, z) and non-spatial attributes (i.e., RGB, intensity, etc.).

❹ `Taxi` is extracted from records of yellow taxi trips in New York City, for the months of Jan 2015 and Jan-mar 2016 [61]. The dataset includes 4 spatial attributes (i.e., pickup and dropoff locations) and 5 numeric attributes like passenger count, trip distance, total charge, etc.

❺ `Tpch` is the most commonly used benchmark for relational DBMS [80]. In this benchmark, we extract 7 numeric attributes from the `lineitem` table to form the multi-dimensional dataset.

❻ `Stock` is a tabular dataset of daily stock prices from 1970 to 2018 [22]. Each record consists of 7 attributes, such as datetime, open price, close price, etc.

Synthetic Datasets. To better evaluate index scalability w.r.t. different data sizes and dimensions, we also sample random points from uniform, normal, and log-normal distributions where the dimension correlation is ignored. We vary the scale factors of each distribution (e.g., the standard deviation of normal distribution) to make sure that the density of generated data is similar. For all synthetic datasets, the default #Points and #Dim. are set to 20 million and 2, respectively.

Range Query Generation. We first randomly sample $S = 100$ points from a given dataset $\mathcal{X}$ as the lower corners of query boxes. Then, for each dimension $i = 1, \cdots, d$, a random width δ_i is added to the lower corner to create the upper corner. For a generated query box R, the selectivity of R is defined as the percentage of points in $\mathcal{X}$ covered by R, i.e.,

$$\mathsf{sel}(R) = \frac{|\{\mathbf{x} \mid \mathbf{x} \in \mathcal{X} \wedge R \text{ covers } \mathbf{x}\}|}{|\mathcal{X}|}. \tag{4}$$

Figure 10 shows the selectivity distributions of the generated range queries across different datasets. We categorize the set of generated query boxes $\mathcal{R}$ based on selectivities into three groups: (a) low selectivity ($\mathsf{sel}(R) \leq 1\%$), (b) middle selectivity ($1\% < \mathsf{sel}(R) \leq 5\%$), and (c) high selectivity ($\mathsf{sel}(R) > 5\%$). For query boxes in each selectivity group, we report the *average* query time in subsequent evaluations.

Extension to Box Geometries. As discussed in Sect. 2.1, *all* learned indexes in this benchmark are designed for point geometries. However, since indexing bounding boxes is essential in many real-world spatial applications, we extend each point-based index to support querying box geometries. Due to the limited availability of large-scale bounding box datasets, we generate boxes for indexing using a similar approach to that used for generating range query boxes. Specifically, for each point in datasets `FourSquare`, `OSM`, and `Toronto3d`, a random width within $[0, W/100]$ is added for each dimension to synthesize a box geometry, where W is the maximum data range.

***k*NN Query Generation.** A kNN query consists of a query point q and a specified result size k. To generate kNN queries, we randomly select $S = 100$ points from each dataset for

each $k \in \{1, 10, 100, 1000, 10000\}$. The default value of k in our evaluation is set to 100.

In the subsequent evaluations, we report the *average* query processing time of different selectivities for range queries and different k for kNN queries. Notably, we do not repeatedly execute each query many times, as this would warm the CPU cache and thus reduce query processing times. This approach aims to better simulate practical index usage scenarios.

5 Experiment results

In this section, we report the evaluation results based on the above benchmark setups. Specifically, we focus on the following 9 sub-questions.

- **Q1: Construction Time**: whether the construction time of learned indexes is as fast as the non-learned baselines (▷ Sect. 5.1);
- **Q2: Space Cost**: whether the space cost of learned indexes is significantly lower than the non-learned baselines (▷ Sect. 5.2);
- **Q3: Range Query Efficiency**: whether the learned indexes can outperform baselines in terms of range query processing (▷ Sect. 5.3);
- **Q4: kNN Query Efficiency**: whether the learned indexes can outperform baselines in terms of kNN query processing (▷ Sect. 5.4);
- **Q5: Scalability**: whether the learned indexes can well scale to datasets of larger sizes and higher dimensions (▷ Sect. 5.5);
- **Q6: Parameter Setting:** how the hyper-parameters of underlying 1-D learned indexes (e.g., ϵ) affect the performance (▷ Sect. 5.6);
- **Q7: Dynamic Update Efficiency:** whether existing learned indexes can support efficient dynamic updates (▷ Sect. 5.7);
- **Q8: Extended Geometries:** whether the learned indexes can efficiently support indexing and querying over extended geometries like bounding boxes (▷ Sect. 5.8);
- **Q9: Explanation of Performance:** why learned indexes are effective (or ineffective) from the hardware perspective (▷ Sect. 5.9).

5.1 Index construction time (Q1)

We first report the index construction time in Table 4. If the construction time exceeds a threshold of 5 hours, it will be terminated, and "N.A." is reported.

As shown in Table 4, all projection-based indexes except `MLI` are comparable with or even faster than the non-learned baselines in terms of index construction time. For example, `ZMI` is $\sim 2\times$ faster than the bulk-loading R-tree index `STR` and is only slower than the simple uniform grid index `UG`. The reasons are twofold. First, the computation of projection functions based on SFC is typically fast by using bit interleaving tricks [56]. Second, we unify the underlying 1-D learned index of these indexes as the PGM-Index, which requires only one pass of data (i.e., $O(N)$ time) to obtain the optimal piecewise linear approximation [25, 54].Compared to `ZMI`, `Wazi` and `LMSFC` take more time for construction due to the extra overhead to learn a better Z-curve based on cost models. For `MLI`, it is less performant because finding reference points via k-means is costly, which occupies $> 95\%$ of the total index building time.

Indexes using grid layouts, i.e., `LISA` and `Flood`, take slightly longer construction time due to the generation of internal equal-depth grid cells. The construction time of `IFI` is about $1.5\times$ of `STR` but much faster than `R*tree` (about $30\times$ faster). This is because we adopt a similar `STR` bulking loading strategy in our implementation of `IFI`. Additionally, the leaf node capacity of `IFI` is generally larger (e.g., 1000) compared to an ordinary R-tree (typically 128), which reduces the cost caused by fitting linear models.

Compared to all the other learned indexes, `RSMI` takes considerably longer time to build ($10^4\times$ slower than `ZMI`). This is due to the deep learning models adopted by `RSMI`, which are significantly more intricate than the algorithmic approaches like PGM-Index. Though accelerating query processing using novel hardware like GPU and TPU is getting popular recently [73], in this benchmark, we focus on the general application scenarios where CPU-based indexes are adopted.

Takeaways. The construction time highly depends on the internal model choices. Indexes based on the PGM-Index are generally as fast as the optimized non-learned baselines; in contrast, indexes that employ deep learning techniques tend to be much slower.

5.2 Index memory cost (Q2)

We then report the memory cost of each index. Notably, for indexes `RSMI`, `kdtree`, `octree`, and `ANN`, we are unable to programmatically retrieve their memory costs at runtime. Thus, we perform heap profiling for these indexes via `gperftools` [31] starting before building the index and ending after the index is constructed. This method for measuring memory cost at runtime is also adopted in a recent benchmark of spatial libraries [66].

Table 5 shows the memory cost evaluation results. As we fix the error threshold ϵ of the underlying PGM-Index to 64, the resulting learned multi-dimensional indexes except `RSMI` share a similar level of memory cost. And all the learned indexes achieve significant improvement of memory overhead compared with popular non-learned indexes (e.g., on `OSM` dataset, `LISA` is $258\times$ smaller than `STR`, $519\times$

Table 4 Index construction time (seconds) on real datasets and synthetic datasets of default configurations. STR is selected as the baseline index to compute the relative ratio. Note that, RSMI only supports 2-D data and fails to terminate construction in 5 hours for dataset OSM.

Index	Data Uniform 20M 2D	Normal 20M 2D	Lognormal 20M 2D	Foursquare 3.7M 2D	Toronto3d 21M 7D	OSM 63M 2D	Taxi 47M 9D	Tpch 100M 8D	Stock 21M 7D
ZMI	1.89	1.63	1.13	0.17	1.09	3.08	0.81	9.31	1.07
	(▼1.76×)	(▼1.99×)	(▼2.85×)	(▼2.76×)	(▼3.0×)	(▼2.25×)	(▼4.96×)	(▼2.75×)	(▼3.96×)
HMI	2.03	2.12	1.26	0.20	1.20	3.41	0.98	10.20	1.20
	(▼1.64×)	(▼1.53×)	(▼2.55×)	(▼2.34×)	(▼2.72×)	(▼2.03×)	(▼4.11×)	(▼2.51×)	(▼3.52×)
SLBRIN	2.49	2.88	1.84	0.33	1.70	5.82	1.60	15.91	2.00
	(▼1.34×)	(▼1.13×)	(▼1.75×)	(▼1.44×)	(▼1.92×)	(▼1.19×)	(▼2.52×)	(▼1.61×)	(▼2.12×)
Wazi	3.26	3.93	4.28	0.67	4.87	12.13	7.28	49.17	8.61
	(▼1.02×)	(▲1.21×)	(▲1.33×)	(▲1.42×)	(▲1.49×)	(▲1.75×)	(▲1.81×)	(▲1.92×)	(▲2.03×)
LMSFC	4.16	4.45	4.54	0.68	5.69	13.79	8.48	60.95	10.68
	(▲1.25×)	(▲1.37×)	(▲1.41×)	(▲1.45×)	(▲1.74×)	(▲1.99×)	(▲2.11×)	(▲2.38×)	(▲2.52×)
MLI	93.64	90.45	84.33	14.51	83.29	209.91	137.40	1776.05	240.49
	(▲28.12×)	(▲27.83×)	(▲26.19×)	(▲30.88×)	(▲25.47×)	(▲30.29×)	(▲34.18×)	(▲69.35×)	(▲56.72×)
IFI	4.88	5.04	5.08	0.71	4.61	10.15	5.95	41.18	9.77
	(▲1.47×)	(▲1.55×)	(▲1.58×)	(▲1.51×)	(▲1.41×)	(▲1.46×)	(▲1.48×)	1.61	(▲2.30×)
RSMI	8978	13017	10311	996.8	N.A.	N.A.	N.A.	N.A.	N.A.
	(▲2696×)	(▲4005×)	(▲3202×)	(▲2121×)					
LISA	7.46	7.44	7.47	1.25	7.55	15.59	11.30	77.09	11.45
	(▲2.24×)	(▲2.29×)	(▲2.32×)	(▲2.66×)	(▲2.31×)	(▲2.25×)	(▲2.81×)	(▲3.01×)	(▲2.70×)

Table 4 continued

Index	Data Uniform 20M 2D	Normal 20M 2D	Lognormal 20M 2D	Foursquare 3.7M 2D	Toronto3d 21M 7D	OSM 63M 2D	Taxi 47M 9D	Tpch 100M 8D	Stock 21M 7D
Flood	5.33	5.43	5.57	0.99	9.81	18.02	13.87	73.50	12.47
	(▲1.60×)	(▲1.67×)	(▲1.73×)	(▲2.11×)	(▲3.00×)	(▲2.60×)	(▲3.45×)	(▲2.87×)	(▲2.94×)
STR	3.33	3.25	3.22	0.47	3.27	6.93	4.02	25.61	4.24
	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)
R*tree	128.92	128.80	120.46	18.22	155.34	389.35	150.22	130.28	200.05
	(▲38.7×)	(▲39.6×)	(▲37.4×)	(▲38.8×)	(▲47.5×)	(▲56.2×)	(▲37.37×)	(▲5.09×)	(▲47.18×)
kdtree	28.61	27.96	29.03	3.87	14.66	30.53	50.75	127.37	130.09
	(▲8.59×)	(▲8.60×)	(▲9.02×)	(▲8.23×)	(▲4.48×)	(▲4.41×)	(▲12.62×)	(▲4.97×)	(▲30.68×)
octree	2.15	2.46	2.77	0.619	N.A.	21.20	N.A.	N.A.	N.A.
	(▼1.55×)	(▼1.32×)	(▼1.16×)	(▲1.32×)		(▲3.06×)			
ANN	21.61	20.43	20.75	3.21	2.99	22.33	34.32	87.94	79.72
	(▲6.49×)	(▲6.29×)	(▲6.44×)	(▲6.83×)	(▼1.09×)	(▲3.22×)	(▲8.54×)	(▲3.43×)	(▲18.80×)
UG	0.73	0.51	0.34	0.07	0.48	1.24	0.62	8.51	0.89
	(▼4.56×)	(▼6.37×)	(▼9.47×)	(▼6.71×)	(▼6.81×)	(▼5.59×)	(▼6.48×)	(▼3.01×)	(▼4.76×)
EDG	4.17	4.16	4.15	0.63	4.53	8.44	5.12	43.93	7.67
	(▲1.25×)	(▲1.28×)	(▲1.29×)	(▲1.34×)	(▲1.39×)	(▲1.22×)	(▲1.27×)	(▲1.72×)	(▲1.81×)

Table 5 Index memory overhead (MiB) on real datasets and synthetic datasets of default configurations, with `STR` chosen as the baseline to compute the relative comparison ratio. We exclude the memory cost for storing pointers to each data point from the overall index size where each pointer occupies 8 bytes on most modern architectures.

Index	Data Uniform 20M 2D	Normal 20M 2D	Lognormal 20M 2D	Foursquare 3.7M 2D	Toronto3d 21M 7D	OSM 63M 2D	Taxi 47M 9D	Tpch 100M 8D	Stock 21M 7D
ZMI	0.02	0.05	0.03	0.03	0.44	1.94	93.19	620.30	159.96
	(▼7374×)	(▼3120×)	(▼5070×)	(▼907×)	(▼1235×)	(▼262×)	(▼6.18×)	(▼7.29×)	(▼5.16×)
HMI	0.03	0.06	0.03	0.05	0.34	2.80	109.70	1027.73	212.19
	(▼6324×)	(▼2571×)	(▼5278×)	(▼661×)	(▼1584×)	(▼182×)	(▼5.25×)	(▼4.40×)	(▼3.89×)
SLBRIN	0.04	0.07	0.03	0.04	0.38	2.47	108.05	668.93	169.49
	(▼3983×)	(▼2271×)	(▼6027×)	(▼702.8×)	(▼1431×)	(▼205.9×)	(▼5.33×)	(▼6.76×)	(▼4.87×)
Wazi	0.02	0.04	0.03	0.03	0.40	1.76	85.70	625.45	132.28
	(▼7862×)	(▼3943×)	(▼5568×)	(▼919.2×)	(▼1351×)	(▼289.37×)	(▼6.72×)	(▼7.23×)	(▼6.24×)
LMSFC	0.02	0.04	0.03	0.03	0.37	1.72	76.89	566.67	121.20
	(▼7921×)	(▼4025×)	(▼6014×)	(▼1008×)	(▼1475×)	(▼295.5×)	(▼7.49×)	(▼7.98×)	(▼6.81×)
MLI	0.04	0.04	0.04	0.05	0.12	0.29	19.58	126.38	18.24
	(▼3863×)	(▼3863×)	(▼3863×)	(▼564×)	(▼4528×)	(▼1749×)	(▼29.41×)	(▼35.78×)	(▼45.25×)
IFI	1.12	1.12	1.12	0.21	2.19	3.51	271.65	1745.95	441.39
	(▼145×)	(▼145×)	(▼145×)	(▼140×)	(▼248×)	(▼145×)	(▼2.12×)	(▼2.59×)	(▼1.87×)
RSMI	21.5	22.4	29.8	21.7	N.A.	N.A.	N.A.	N.A.	N.A.
	(▼7.55×)	(▼7.24×)	(▼5.44×)	(▼1.38×)					
LISA	0.53	0.53	0.53	0.09	0.47	1.97	131.18	1306.94	205.32
	(▼305×)	(▼305×)	(▼305×)	(▼321×)	(▼1156×)	(▼258×)	(▼4.39×)	(▼3.46×)	(▼4.02×)

Table 5 continued

Index	Data Uniform 20M 2D	Normal 20M 2D	Lognormal 20M 2D	Foursquare 3.7M 2D	Toronto3d 21M 7D	OSM 63M 2D	Taxi 47M 9D	Tpch 100M 8D	Stock 21M 7D
Flood	0.33	0.40	0.40	0.28	4.45	4.63	117.05	856.44	197.94
	(▼492×)	(▼406×)	(▼406×)	(▼106×)	(▼122×)	(▼110×)	(▼4.92×)	(▼5.28×)	(▼4.17×)
STR	162.2	162.2	162.2	29.9	543.4	508.9	575.9	4522	825.4
	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)	(1.0×)
R*tree	295.5	292.5	287.5	56.0	945.52	1021.9	2162	6912	1712
	(▲1.82×)	(▲1.80×)	(▲1.77×)	(▲1.88×)	(▲1.74×)	(▲2.01×)	(▲3.75×)	(▲1.53×)	(▲2.07×)
kdtree	120.15	118.39	115.86	16.70	192.01	200.35	275.55	2018.75	378.62
	(▼1.35×)	(▼1.37×)	(▼1.40×)	(▼1.79×)	(▼2.83×)	(▼2.54×)	(▼2.09×)	(▼2.24×)	(▼2.18×)
octree	1516.1	1542.8	1570.4	266.5	N.A.	4583.0	N.A.	N.A.	N.A.
	(▲9.35×)	(▲9.51×)	(▲9.68×)	(▲8.93×)		(▲9.01×)			
ANN	127.72	122.88	125.74	18.92	205.83	222.23	286.52	2143.13	416.87
	(▼1.27×)	(▼1.32×)	(▼1.29×)	(▼1.58×)	(▼2.64×)	(▼2.29×)	(▼2.01×)	(▼2.11×)	(▼1.98×)
UG	0.22	0.22	0.22	0.08	0.28	1.43	6.41	45.87	160.02
	(▼731×)	(▼731×)	(▼731×)	(▼360×)	(▼1941×)	(▼356×)	(▼89.87×)	(▼98.58×)	(▼5.16×)
EDG	0.22	0.22	0.22	0.08	0.28	1.43	6.41	45.87	160.02
	(▼731×)	(▼731×)	(▼731×)	(▼360×)	(▼1941×)	(▼356×)	(▼89.87×)	(▼98.58×)	(▼5.16×)

smaller than R*tree, and ∼ 102× smaller than kdtree). For projection-based indexes, difference choices of projection functions also influence space cost. Recent theoretical studies [24] indicate that the size of a PGM-Index is determined by distribution characteristics of the mapped values.

Grid indexes like UG and EDG also exhibit low space cost (sometimes outperforming learned indexes). This is because these grid indexes only require to store the partition boundaries on each dimension. However, they generally perform badly on query processing, especially for skewed and high-dimensional datasets, which will be discussed in Sect. 5.3.

For augmentation-based indexes, IFI also achieves remarkable memory cost as its leaf node capacity is much larger (i.e., 1000) than an ordinary R-tree (typically 128), thus requiring much fewer tree nodes to store. As for RSMI, though its index construction time is significantly higher than that of other indexes (9124× larger than ZMI on dataset Lognormal, see Table 4), the trained model is generally more compact for storage and not sensitive to the size of various datasets (about 20MB for all datasets). Thus, an ideal use case of RSMI is to perform offline training using machines equipped with powerful GPUs and deploy the trained models for subsequent query processing.

Takeaways. All evaluated learned indexes achieve a much more **compact** structure compared to conventional indexes with a sacrifice of affordable index construction cost.

5.3 Range query processing (Q3)

We then present the range query evaluation results using the queries generated in Sect. 4.2. Figure 11 shows the *average* query processing time w.r.t. different levels of query selectivities: (a) low selectivity ($\mathsf{sel}(R) \leq 1\%$), (b) middle selectivity ($1\% < \mathsf{sel}(R) \leq 5\%$), and (c) high selectivity ($\mathsf{sel}(R) > 5\%$).

Among the non-learned baselines, STR and EDG are the fastest indexes on range query processing. In comparison, learned indexes IFI, Flood, and LISA consistently achieve better performance, regardless of data skewness query selectivity, and dimensionality. Not surprisingly, learned indexes excel with less selective queries (e.g., $\mathsf{sel(R)} < 0.5\%$). For example, on Lognormal, Flood is 2.19× faster than STR when query selectivity is less than 1%, whereas the speed-up ratio decreases to 1.31 when the selectivity ratio increases to 48%. This is because, the pruning power of learned models is much more significant for small query ranges, where in the worst case all indexes, both learned and non-learned, perform no better than a naive linear scan.

RSMI does not show satisfactory performance, particularly on non-uniform data. This is because RSMI is designed as a disk-based index structure, aiming to minimize the page access. Additionally, RSMI also supports *approximate* range query processing, which is about an order of magnitude faster than the exact query processing when query selectivity is lower than 5%. The performance of MLI is also not satisfactory, as its metric-based projection function (Eq. (1)) cannot effectively encode spatial locality information, making it unsuitable for orthogonal range query processing Euclidean spaces.

An interesting observation is that ZMI, the first learned multi-dimensional index, is often regarded as a weak baseline in previous studies [47, 58, 69]. However, our evaluation reveals that the performance of ZMI is not significantly worse and is comparable to that of STR, especially considering its low construction time and memory cost. The major reason is that the original implementation of ZMI [81] adopts a coarse pruning strategy where the whole range of [Z(min_corner), Z(max_corner)] should be searched. In this work, we use the BigMin algorithm [71] to divide a query box into fine-grained candidate Z-value ranges and perform a learned index-based search over the points sorted by Z-values, leading to a significant performance improvement.

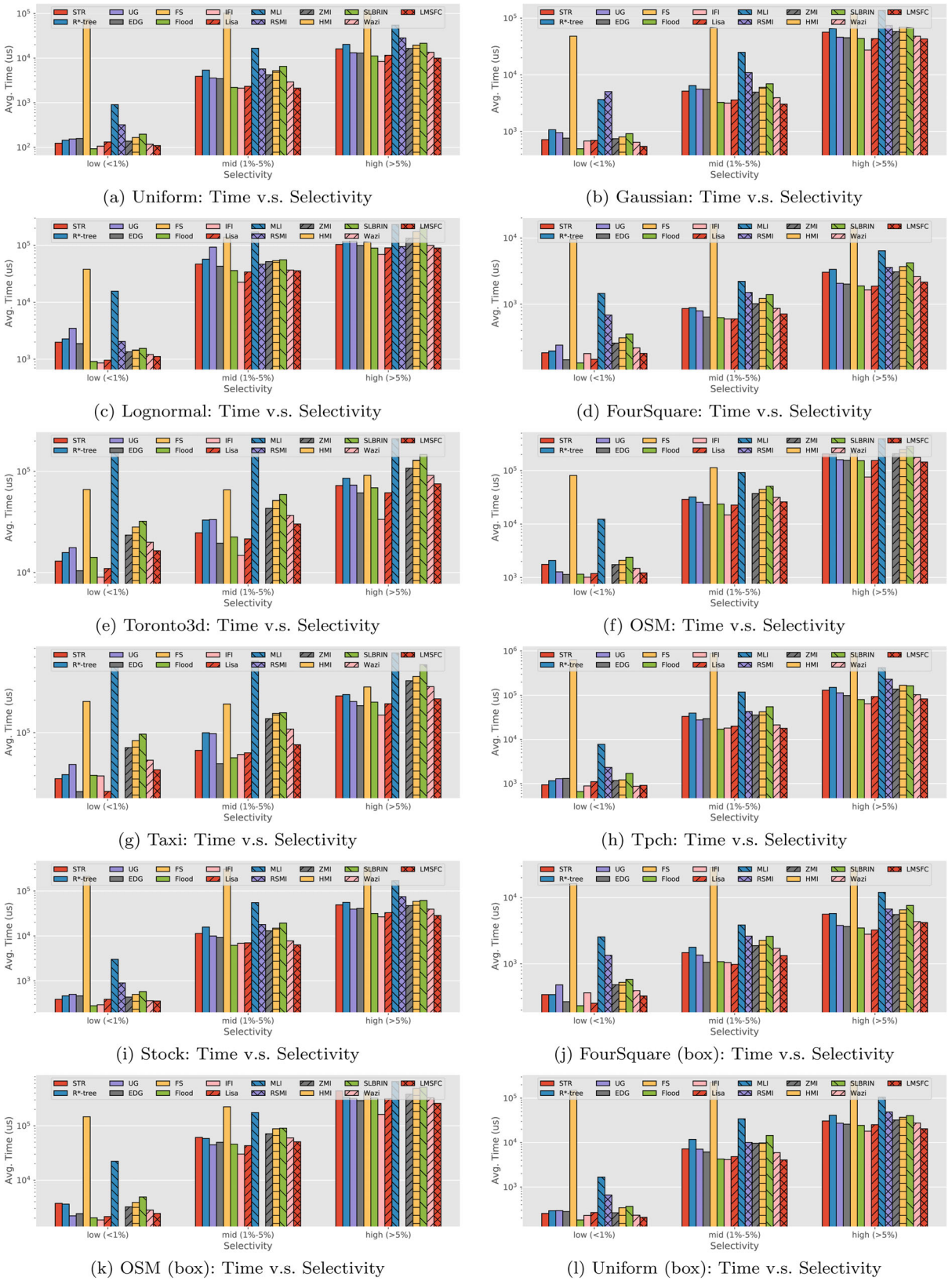
Similar approaches HMI and SLBRIN based on Hilbert-curve and GeoHash do not outperform ZMI in range query processing. The reasons are twofold: (a) Z-address computation is much more efficient on modern hardware, and (b) the BigMin algorithm can well prune unnecessary candidates. Additionally, two adaptive versions of ZMI, i.e., Wazi and LMSFC, consistently outperform ZMI by up to 1.25× and 1.38× in range query processing, respectively, as they can better adapt to skewed data and query workloads.

Takeaways. Learned indexes IFI, Flood, and LISA can robustly outperform the non-learned baselines across all levels of query selectivity. The speedup ratio is especially significant for **uniformly** distributed datasets and **low-selectivity** queries. On the other hand, other indexes like ZMI, MLI, and RSMI cannot **systematically** outperform non-learned indexes.

5.4 kNN query processing (Q4)

We further evaluate the efficiency of kNN query processing three synthetic datasets and three spatial-related real datasets. For dataset Toronto3d, only geographical attributes are considered (i.e., x, y, z). Other datasets, such as Tpch and Stock, are excluded because kNN queries based on L2 distance are less meaningful and can be easily dominated by attributes with larger value domains. Figure 12 reports the average kNN query processing time against varying $k \in \{1, 10, 100, 1000, 10000\}$.

Among non-learned indexes, the two kd-tree variants, kdtree and ANN, perform best for $k \leq 100$, while R-tree variants, STR and R*tree, become more efficient as k increases beyond 100. Unlike range queries, where learned indexes generally outperform non-learned baselines, MLI,



(a) Uniform: Time v.s. Selectivity

(b) Gaussian: Time v.s. Selectivity

(c) Lognormal: Time v.s. Selectivity

(d) FourSquare: Time v.s. Selectivity

(e) Toronto3d: Time v.s. Selectivity

(f) OSM: Time v.s. Selectivity

(g) Taxi: Time v.s. Selectivity

(h) Tpch: Time v.s. Selectivity

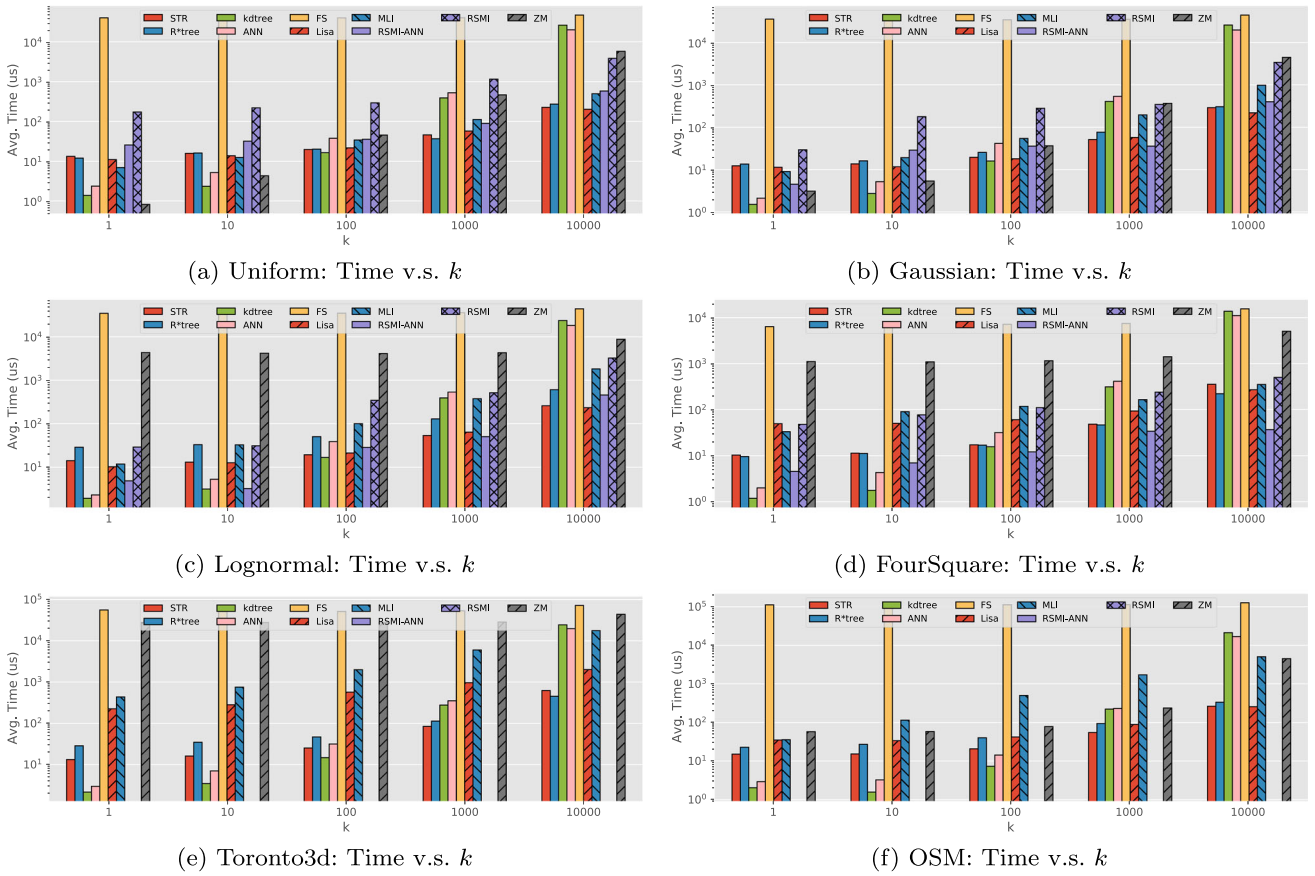
(i) Stock: Time v.s. Selectivity

(j) FourSquare (box): Time v.s. Selectivity

(k) OSM (box): Time v.s. Selectivity

(l) Uniform (box): Time v.s. Selectivity

Fig. 11 Range query evaluation results on three synthetic datasets, six real datasets, and three bounding box datasets (generated based on real datasets), where the y-axis and x-axis refer to the *average* query processing time and *average* selectivity of test queries, respectively



(a) Uniform: Time v.s. k

(b) Gaussian: Time v.s. k

(c) Lognormal: Time v.s. k

(d) FourSquare: Time v.s. k

(e) Toronto3d: Time v.s. k

(f) OSM: Time v.s. k

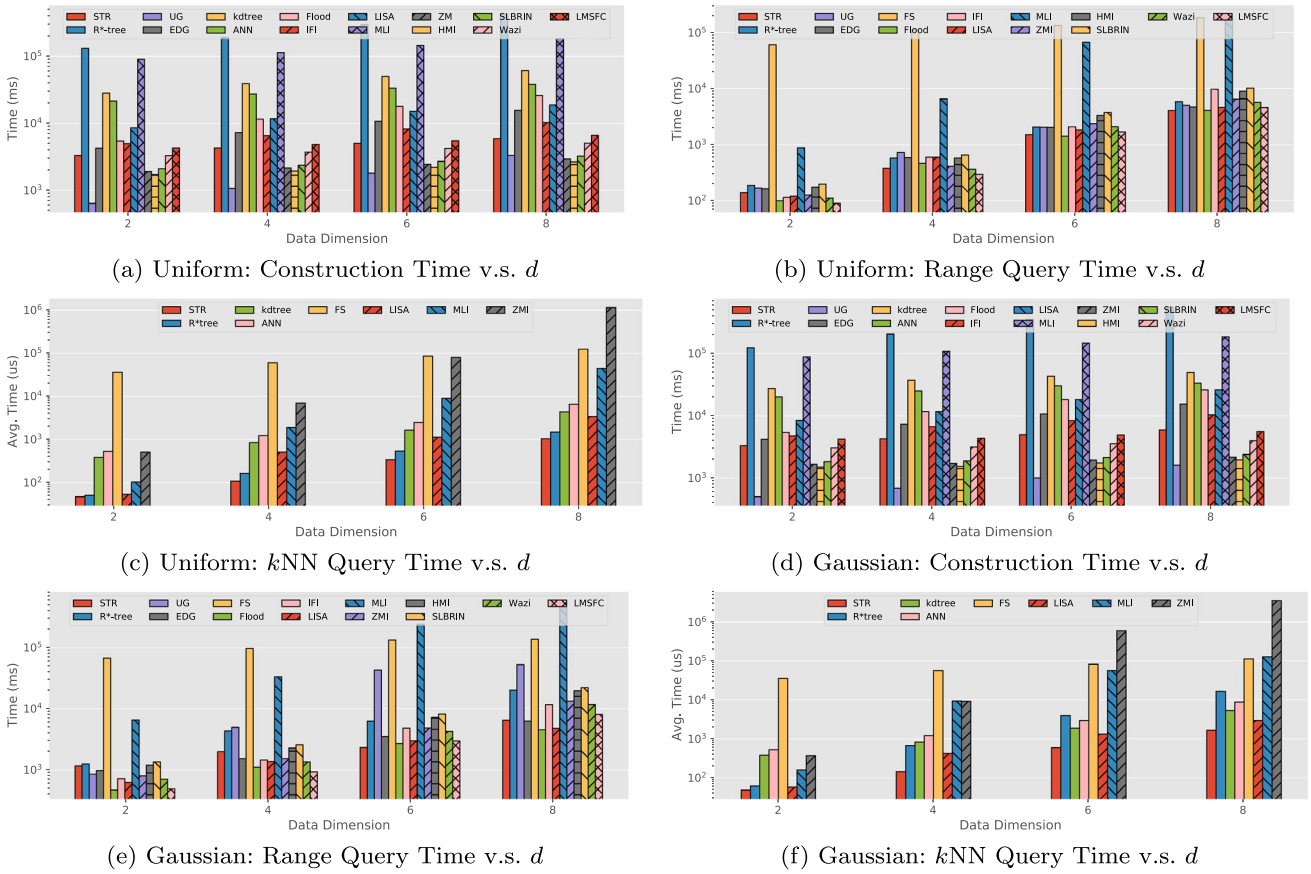
Fig. 12 kNN query evaluation results on three synthetic datasets and three spatial-related real datasets. The y-axis refers to the *average* query processing time for each k

`LISA`, `ZMI`, and `RSMI` can outperform the highly optimized R-tree and kd-tree libraries on *some* datasets and *some* k. For example, on synthetic datasets, `MLI` is $1.1 \sim 1.3\times$ faster than `STR` for $k \leq 10$; however, as k increases to 10,000, `MLI` can become up to $3.6\times$ slower than `STR`. Unfortunately, no multi-dimensional learned index consistently outperforms the best non-learned baselines across all three real-world datasets.

The primary reason is that, most of the existing learned indexes process kNN queries by repeatedly invoking *range queries* with progressively increased search radius until the nearest k results are found, which is highly sensitive to the initial search range setting. In the worst case, such an approach may require expanding the search radius from a small range up to the entire data space, resulting in numerous unnecessary range queries before locating the kNN results. Furthermore, unlike R-tree and kd-tree variants, which rely on recursive space partitioning, it is challenging to incorporate local aggregation information (e.g., range counts) within learned index structures, making it difficult to effectively prune candidates during index traversal.

We also evaluate the approximate kNN query processing, which is available only for `RSMI` among existing learned indexes, referred to as `RSMI-ANN` in Fig. 12. `RSMI-ANN` significantly improves the kNN query processing efficiency by up to three orders of magnitudes compared to the exact `RSMI`. Additionally, following [69], we also evaluate the recall of approximate kNN results, defined as the ratio of retrieved points that match the actual kNN results. On six datasets, `RSMI-ANN` and conventional ANN mthods based on kd-trees exhibit average recalls of 89% and 82%, respectively, highlighting `RSMI-ANN` as a promising approximate kNN index, especially for large values of k. However, as most existing multi-dimensional learned indexes are not particularly optimized for kNN and ANN queries, this benchmark does not include popular ANN methods like HNSW [52] and product quantization [39].

Takeaways. Different from range queries, the nature of progressive range search prevents efficient kNN query processing for existing multi-dimensional learned indexes. The kNN processing efficiency for learned indexes **cannot** systematically outperform non-learned baselines, especially on datasets of high dimensions.



(a) Uniform: Construction Time v.s. d

(b) Uniform: Range Query Time v.s. d

(c) Uniform: kNN Query Time v.s. d

(d) Gaussian: Construction Time v.s. d

(e) Gaussian: Range Query Time v.s. d

(f) Gaussian: kNN Query Time v.s. d

Fig. 13 Scalability evaluation w.r.t. dimension d. The range query selectivity is in $[10^{-5}, 10^{-2}]$, and $k = 1000$ for kNN queries

5.5 Scalability evaluation (Q5)

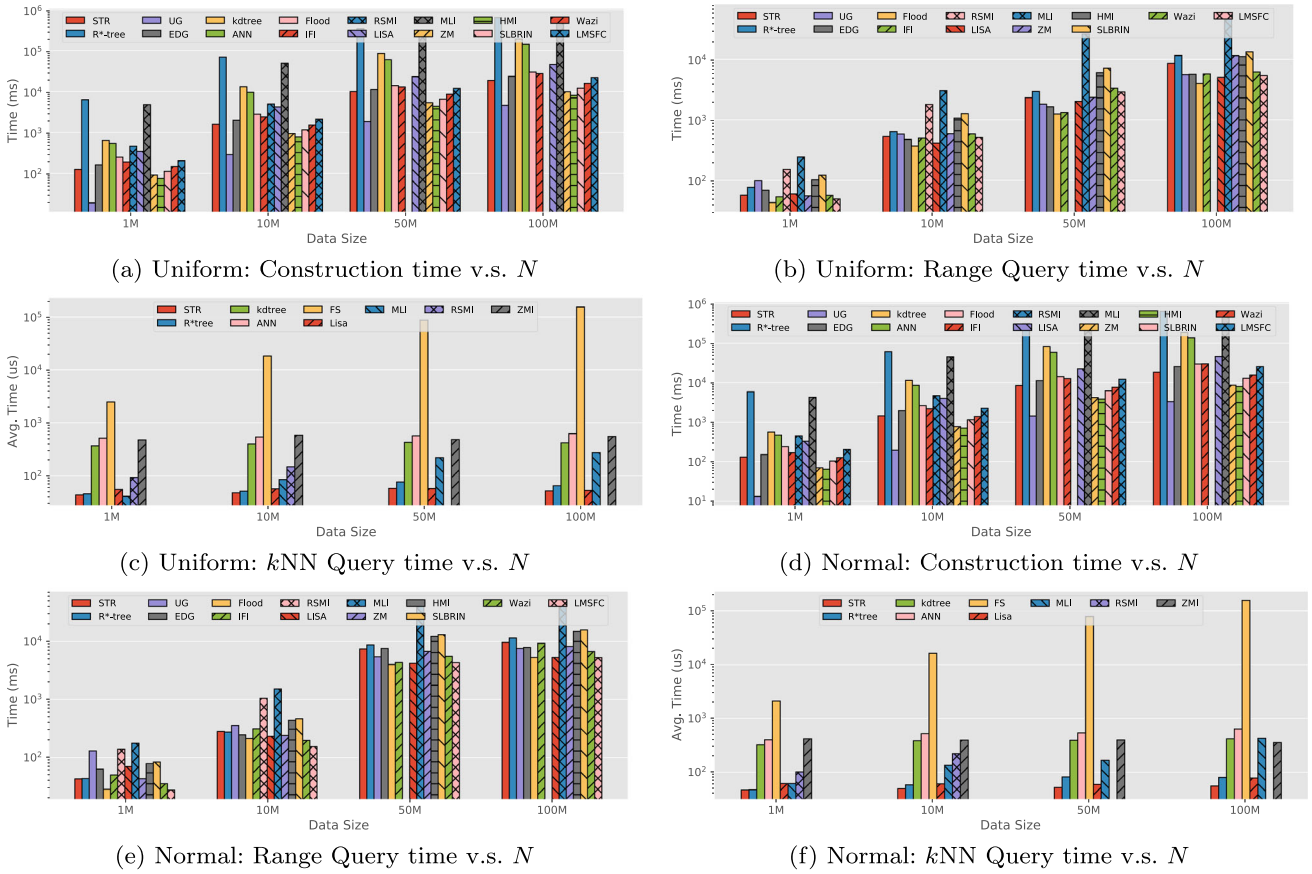
We then study the performance of index construction and query processing when scaling the size and dimension of datasets.

Figure 13 presents the scalability evaluation results by varying data dimension $d \in \{2, 4, 6, 8\}$ for two synthetic datasets. The construction time for all compared indexes grows as d increases. For all grid-based learned indexes such as `Flood`, we set the partition number for each dimension to $\sqrt[d]{N/B}$, which avoids the exponential growth of the grid size w.r.t. dimension d, ensuring each grid cell contains approximately B points. Notably, SFC-based indexes, such as `ZMI`, `HMI`, `Wazi`, and `LMSFC`, are insensitive to the increase of d, since increasing d slightly slows down the computation of Z-addresses, which constitutes only a small fraction of the overall index construction time.

For query processing efficiency, the results are similar to those discussed in Sects. 5.3 and 5.4. Specifically, `Flood` and `LISA` can robustly achieve better range query performance when d scales. In terms of kNN query processing, `MLI` is the best learned index on low dimensional data ($d \leq 3$) as `MLI` can be viewed as a learned version of "iDistance+B-tree", which encodes the distance information and thus is more suitable for kNN query processing. However, the performance of `MLI` is worse than that of `Flood` and `LISA` when d increases. This can be attributed to the limitations of the projection function based on Euclidean distance (i.e., Eq. (1)), which cannot well distinguish the differences of two points to a query point when d is large [1]. Besides, the cost of computing Eq. (1) also grows with d, becoming a non-negligible part of the total kNN query processing overhead.

Figure 14 also presents the scalability evaluation results by varying the size of synthetic datasets from 1M to 100M. The index construction time for learned indexes typically scales with the increase in data size N, considering that all learned indexes require sorting the data based on some attributes and then training models on the sorted data. `RSMI` fails to terminate training for data sizes larger than 50M (i.e., exceeding 5 hours). For range queries with fixed selectivity, the processing time of all indexes increases as more results need to be reported; however, the relative performance ratio between learned and non-learned indexes remains consistent with the default setting (Fig. 11).

Takeaways. Existing multi-dimensional learned indexes can generally scale to large datasets (up to 100M) and a moderate dimensions (up to 8), covering a wide range of real-world scenarios from geographical applications to RDBMS filtering.



(a) Uniform: Construction time v.s. N

(b) Uniform: Range Query time v.s. N

(c) Uniform: kNN Query time v.s. N

(d) Normal: Construction time v.s. N

(e) Normal: Range Query time v.s. N

(f) Normal: kNN Query time v.s. N

Fig. 14 Scalability evaluation w.r.t. data size N. The range query selectivity is in $[10^{-4}, 10^{-2}]$, and $k = 1000$ for kNN queries

Specifically, indexes `Flood` and `LISA` can robustly outperform conventional indexes in range query processing time. Index `RSMI` fails to scale to datasets of size up to 50M as the model training cannot be terminated within 5 hours.

5.6 Effects of error parameter (Q6)

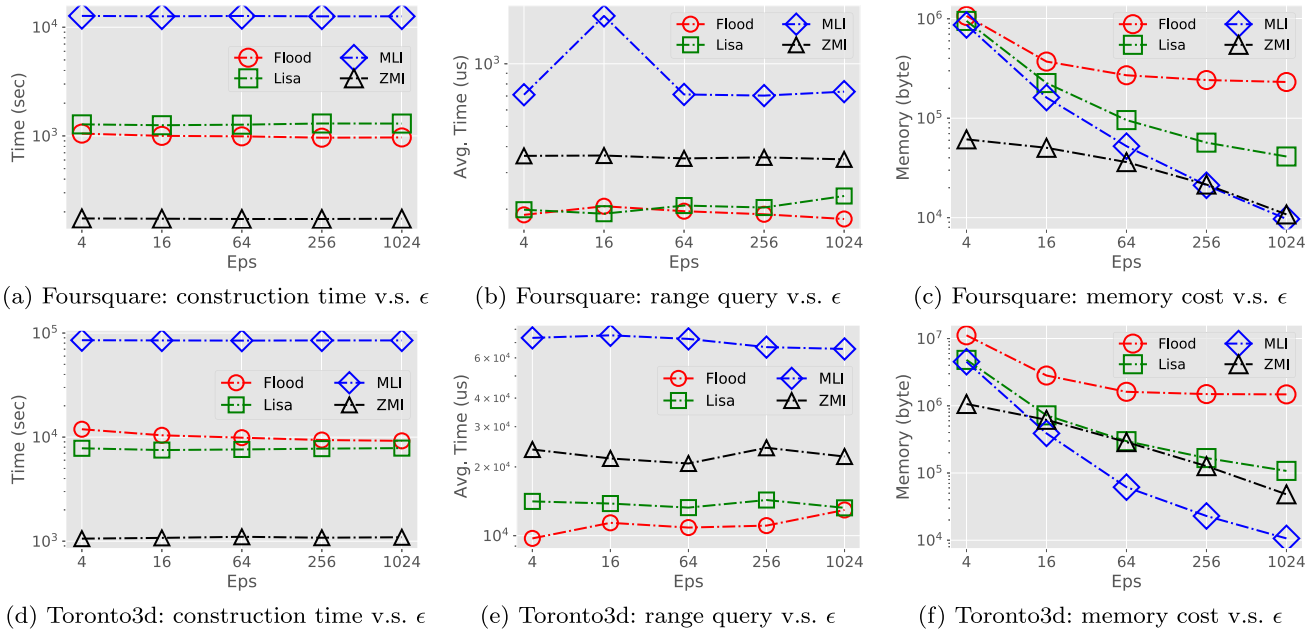
In this section, we study the effect of error constraints for the indexes internally using PGM-Index as the 1-D learned index implementation (e.g., `Flood`, `LISA`, `MLI`, and `ZMI`). Other indexes that also internally employ PGM-Index, such as `Wazi` and `LMSFC`, are not reported as the results are similar. We adopt a generic PGM-Index implementation, which involves two error parameters: the internal search error and the last-mile error. For a given last-mile error ϵ, the internal error parameter is automatically configured by a recent work PGM++ [49], aimed at minimizing expected query costs. Figure 15 shows the index construction time, range query processing time, and index memory footprint when varying the last-mile error parameter ϵ within the range [4, 1024] on two real datasets.

The index construction time is generally insensitive to ϵ as the optimal piece-wise linear fitting algorithm [62] adopted by PGM-Index requires only one pass of data (i.e., $O(N)$ time), contributing a relatively minor part compared to the overhead from data partitioning and projection function computation.

For range query processing, unlike 1-D learned indexes, the relationship between query time and ϵ is not monotonic and hard to predict. The reason is that, a smaller ϵ reduces the overhead of a single point lookup; however, the most costly part of range query processing is whether the learned models and the underlying data layout can effectively filter unnecessary results.

Regarding space cost, our findings align with the 1-D case reported in [25], where decreasing ϵ enlarges the space overhead as more line segments are required to satisfy the error constraint. According to a recent theoretical analysis of PGM-Index [24], the space cost for a PGM-Index with error parameter ϵ is bounded by $O(N/\epsilon^2)$, consistent with the memory cost evaluation results as shown in Fig. 15.

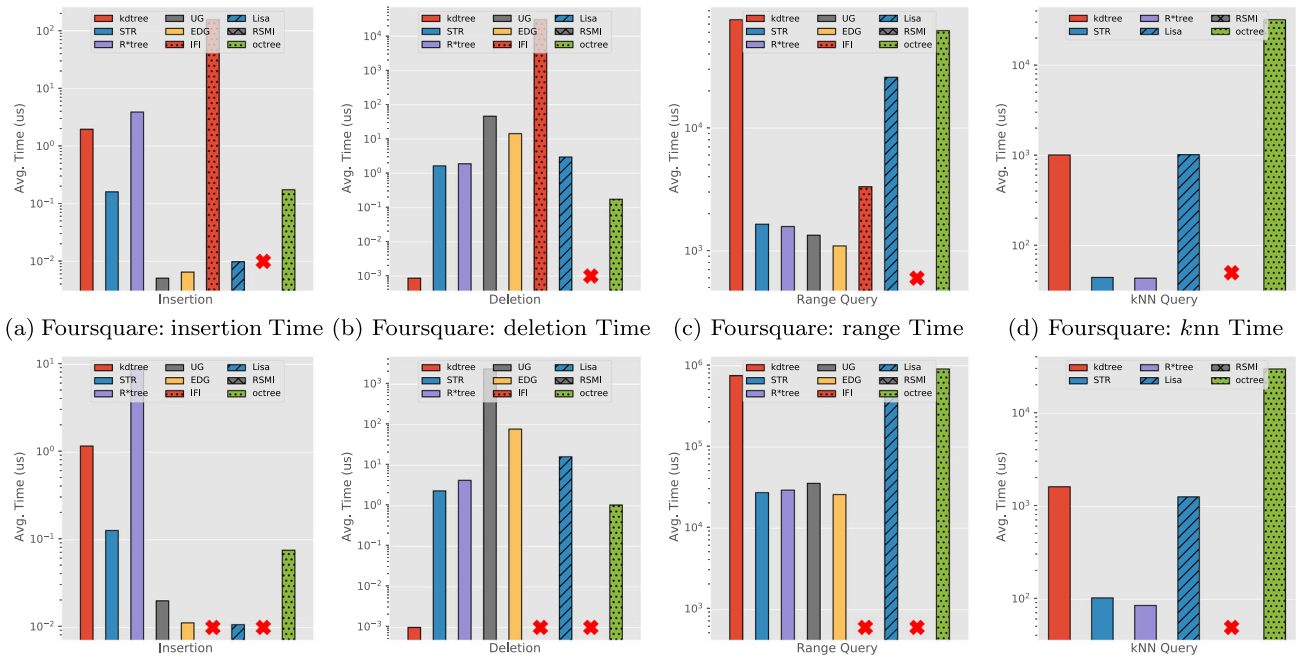
Takeaways. For multi-dimensional learned indexes that internally employ PGM-Index, both index construction time and query processing time are generally **insensitive** to the choice of ϵ, differing from the 1-D case. In practice, we suggest $\epsilon = 64$ for all the learned indexes internally utilizing PGM-Index, as this value effectively balances index



(a) Foursquare: construction time v.s. ϵ (b) Foursquare: range query v.s. ϵ (c) Foursquare: memory cost v.s. ϵ

(d) Toronto3d: construction time v.s. ϵ (e) Toronto3d: range query v.s. ϵ (f) Toronto3d: memory cost v.s. ϵ

Fig. 15 Evaluation results w.r.t. the last-mile search error parameter ϵ (for multi-dimensional learned indexes that are based on PGM-Index) on two real datasets `Foursquare` and `Toronto3d`



(a) Foursquare: insertion Time (b) Foursquare: deletion Time (c) Foursquare: range Time (d) Foursquare: $k$nn Time

(e) Toronto3d: insertion Time (f) Toronto3d: deletion Time (g) Toronto3d: range Time (h) Toronto3d: $k$nn Time

Fig. 16 Data update evaluation results. Note that, `RSMI` fails to terminate on both datasets and `IFI` fails to terminate on the `Toronto3d` dataset

construction cost, memory overhead, and query processing efficiency.

5.7 Index update efficiency evaluation (Q7)

In this section, we evaluate the update efficiency for learned indexes that support dynamic operations (i.e., `LISA`, `RSMI`, and `IFI`). For each dataset, we first construct an index using the initial 80% of the points. We then sequentially insert the remaining 20% of points into the index while randomly removing 20% of the existing points, similar to the setting in [69].

Figure 16 presents the average insertion and deletion efficiency on two real-world datasets `FourSquare` and

Toronto3d. From the results, we observe that the learned indexes marked as updatable (IFI, LISA, and RSMI) generally exhibit lower performance than conventional index structures regarding update efficiency. Moreover, during the data updates, we randomly generate 20 range queries and kNN queries at different timestamps to evaluate the query efficiency under a dynamic environment. The query generation process follows the same way described in Sect. 4.2. In contrast to the previous findings observed under a *static* environmental condition, our evaluation results indicate that, with regard to the efficiency of range query and kNN query processing, learned indexes generally incur higher costs compared to non-learned baselines.

The reason is that current updatable learned indexes on multi-dimensional data usually employ simple strategies to accommodate dynamic updates like setting a threshold of index retraining, introducing non-negligible cost. Moreover, existing techniques for designing updatable 1-D learned index (e.g., ALEX [20] and LIPP [84]) cannot be seamlessly applied to handle the case of multi-dimensional databases. This is because, as previously mentioned, a multi-dimensional learned index comprises two primary components: data layouts and learned models. Consequently, dynamic operations on multi-dimensional learned indexes necessitate efficient updates to both the underlying data layouts (e.g., grids or partitions) and the learned models (e.g., PGM-Index).

Takeaways. Among the *few* learned indexes that are capable of handling dynamic operations, their efficiency in terms of insertion, deletion, and dynamic query processing typically falls short when compared to conventional indexes, such as R-tree and kd-tree variants.

5.8 Performance on extended geometries (Q8)

The previously presented experimental results and discussions focus on *point* geometries, which are generally adopted by most existing multi-dimensional learned indexes. In this section, we evaluate the query processing efficiency when extended to *box* geometries. We synthesize bounding box datasets by augmenting three datasets, Uniform, FourSquare, and OSM, where the details can be found in Sect. 4.2. The queries used are the same range queries as point queries but followed with an extra spatial predicate Q covers $\mathbf{R}$, ensuring the candidate box R is fully covered by query box Q.

Figure 11j–l presents the range query processing results. For learned indexes, querying box geometries takes $1.73\times \sim 2.19\times$ longer time when compared to querying the corresponding point datasets. Additionally, indexes that perform well on point datasets, like Flood and IFI, also achieves better performance on box geometries. For example, Flood is $1.81\times$ faster than STR and $1.79\times$ faster than R*Tree. The core reason is that, in our implementation, querying over box geometries will be decomposed and transformed into several point range queries. We also perform a regression study to fit a linear function $t_{\text{box}} = a \cdot t_{\text{point}} + b$. The resulting coefficient of determination $R^2 = 0.9824$, demonstrating a strong linear correlation.

While we do not position this as our contribution, we implement the support of querying box geometries for multi-dimensional learned indexes. This feature enables existing work to index more complex geometries, such as polygons, which is left for future work.

Takeaways. By augmenting three datasets—Uniform, FourSquare, and OSM—we found that querying box geometries takes $1.73\times$ to $2.19\times$ longer than for point datasets. Notably, efficient indexes like Flood and IFI also perform well with box queries. A regression analysis indicates a strong linear correlation between processing times for box and point queries, with $R^2 = 0.9824$.

5.9 Deep dive into learned index performance (Q9)

To gain deeper insights into the performance of learned indexes, we conduct a comprehensive evaluation of CPU performance counter statistics during the processing of range queries. Specifically, Table 6 presents metrics including the instructions per cycle (IPC), total cache reference count (#CR), cache miss ratio (CMR%), total CPU branch count (#BR), and branch prediction miss ratio (BMR%). The hardware performance counters are collected by accessing the Intel performance monitor unit (PMU) via code-level profiling.

To accurately assess the CPU overhead associated with learned indexes, we select a set of less selective range queries (with sel $\leq 0.1\%$) to minimize the impact of additional overhead from reporting query results. Furthermore, to shed light on the relationship between query processing efficiency and CPU performance counter statistics, we mark the cell colors of Table 6 from the color map ■■■■■. Colors on the left side of the color map (e.g., dark red ■) indicate the longest query processing times (ranking among the three slowest), while colors on the right side (e.g., dark green ■) represent the shortest processing times (ranking among the two fastest).

Learned indexes IFI, LISA, Flood, and LMSFC generally exhibit lower query processing costs, as indicated by their green cell colors in Table 6. It is evident that these relatively performant indexes consistently achieve smaller values across all four CPU performance counter statistics CR, CMR, BR, and BMR. For example, on synthetic dataset Normal, Flood produces 82 million total cache references with a cache miss rate of 30.09%, both of which are nearly the lowest among all compared methods (in the same column).

In terms of IPC, it is interesting to note that, indexes based on space-filling curves, such as ZMI, HMI, Wazi,

Table 6 CPU performance counter statistics (obtained by PMU) when processing range queries on real and synthetic datasets of default configurations. IPC, #CR, CMR%, #BR, BMR% refer to the instructions per cycle, count of total cache references, cache miss rate, count of CPU branches, and CPU branch predication miss rate, respectively. The cell colors from left to right represent range query processing costs ranging from high to low

Index	Uniform	Normal	Lognormal	Foursquare	Toronto3d	OSM
	IPC #CR (CMR%) #BR (BMR%)	IPC #CR (CMR%) #BR (BMR%)	IPC #CR (CMR%) #BR (BMR%)	IPC #CR (CMR%) #BR (BMR%)	IPC #CR (CMR%) #BR (BMR%)	IPC #CR (CMR%) #BR (BMR%)
ZMI	2.15 194M (30.86%) 873M (0.11%)	2.41 126M (29.51%) 484M (0.10%)	2.07 73M (36.48%) 301M (0.12%)	2.93 48M (13.35%) 177M (0.10%)	2.48 248M (49.21%) 1.01B (0.08%)	2.27 563M (56.79%) 1.94B (0.03%)
HMI	2.08 228M (35.62%) 1392M (0.10%)	2.37 137M (30.67%) 823M (0.11%)	2.05 99M (36.32%) 542M (0.10%)	2.88 61M (15.77%) 298M (0.12%)	2.51 368M (44.50%) 1.45B (0.09%)	2.16 701M (52.15%) 2.62B (0.06%)
SLBRIN	2.01 232M (32.77%) 1.41B (0.13%)	2.14 141M (30.19%) 627M (0.11%)	1.99 87M (35.70%) 398M (0.09%)	2.89 62M (12.93%) 245M (0.10%)	2.36 291M (46.52%) 1.20B (0.07%)	2.24 594M (56.39%) 2.51B (0.05%)
Wazi	2.13 180M (31.25%) 820M (0.10%)	2.52 125M (32.19%) 465M (0.11%)	2.19 72M (41.54%) 297M (0.09%)	2.88 37M (10.43%) 159M (0.10%)	2.49 226M (47.53%) 1.08B (0.08%)	2.31 525M (49.81%) 2.06B (0.07%)
LMSFC	2.29 182M (32.14%) 782M (0.11%)	2.50 114M (30.07%) 393M (0.11%)	2.23 71M (37.51%) 274M (0.12%)	2.95 33M (10.08%) 150M (0.11%)	2.33 216M (43.79%) 995M (0.09%)	2.29 496M (50.03%) 1.72B (0.06%)
MLI	1.12 479M (44.29%) 1.16B (6.52%)	1.09 311M (45.78%) 785M (7.09%)	1.04 302M (52.03%) 738M (3.22%)	1.39 109M (29.94%) 257M (1.33%)	1.13 2.01B (53.71%) 3.86B (4.04%)	1.08 1.28B (57.60%) 2.85B (6.23%)
IFI	0.85 139M (40.58%) 179M (1.28%)	0.83 106M (48.62%) 286M (0.95%)	0.81 50M (49.17%) 69M (1.49%)	1.59 39M (25.51%) 119M (0.39%)	0.84 209M (55.83%) 487M (1.25%)	0.80 337M (52.59%) 412M (0.47%)
LISA	1.19 132M (42.29%) 397M (0.87%)	1.14 85M (34.33%) 242M (1.27%)	1.22 46M (41.95%) 130M (0.89%)	1.27 32M (19.17%) 98M (1.05%)	1.33 189M (51.67%) 526M (2.15%)	0.88 361M (54.30%) 1.01B (0.35%)
Flood	1.41 129M (41.61%) 363M (0.67%)	1.26 82M (30.09%) 210M (0.86%)	1.09 47M (43.78%) 116M (0.71%)	1.36 31M (16.09%) 77M (0.69%)	1.25 165M (50.92%) 405M (2.09%)	1.05 357M (55.62%) 995M (0.31%)

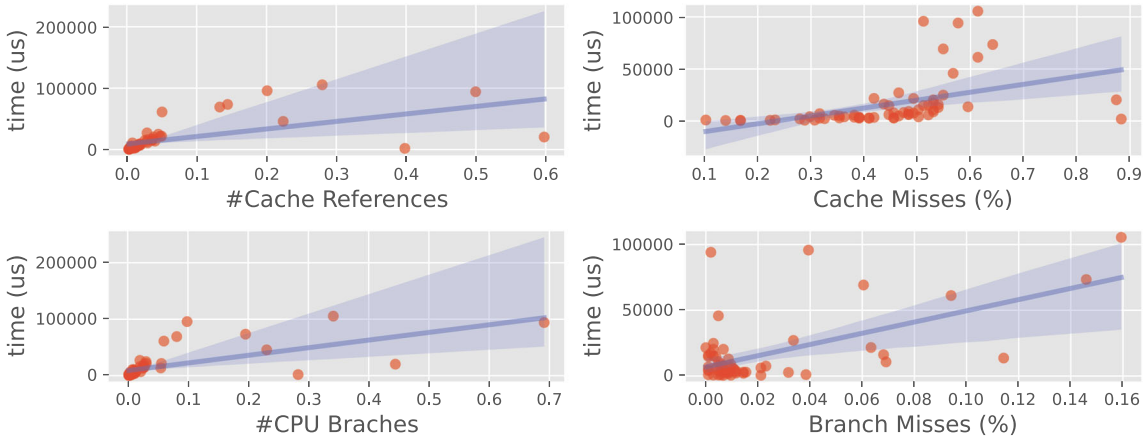


Fig. 17 Range query processing time w.r.t. different CPU performance counter statistics, i.e., #Cache References (CR), Cache Miss Rate (CMR), #CPU Branches (BR), and Branch Miss Rate (BMR)

and LMSFC, typically exhibit a relatively higher IPC value. This can be explained as these internal learned indexes are much smaller (confirmed by Table 5), thus potentially reducing overheads caused by moving model data from memory to CPU. From another angle, the performant indexes like IFI, LISA, and Flood are highly *memory-bound* with IPC fewer than 1. This motivates studies to improve cache utilization for multi-dimensional learned indexes by designing cache-friendly model structures and data layouts.

We then visualize the relationship between range query processing cost against CPU performance counters in Fig. 17. A positive correlation can be individually observed for each CPU performance counter. To further quantitatively depict the correlation between query processing time (the dependent variable y) and the four CPU performance counter statistics (the independent variables x_1: CR, x_2: CMR, x_3: BR, and x_4: BMR), we conduct a linear regression analysis to fit the model $y = w_0 + w_1x_1 + w_2x_2 + w_4x_4$. Notably, we omit the feature x_3 (BR) due to its high linear correlation with x_1 (CR). The

Table 7 Regression results. The dependent variable is the range query processing time and independent variables are #CR, CMR%, and BMR%. A robust linear regression with Huber loss [37] is performed to avoid the influence of outliers

Variable	Coef	Std Err	Z	P>|Z|
const	-6865.3799	2559.049	-2.683	0.007
#CR	1.747e+05	7731.129	22.594	0.000
CMR%	2.098e+04	6306.733	3.327	0.001
BMR%	2.576e+05	2.12e+04	12.161	0.000

results are shown in Table 7 where each independent variable excluding the constant variable is positively correlated to the query processing time with high significance ($P < 0.01$).

The linear regression analysis confirms a general positive linear correlation between query processing costs and the values of the four CPU performance counters. However, this does not imply that lower values in any single statistic (e.g., CPU cache miss rates) necessarily lead to shorter query processing times. For example, for ZMI on the Lognormal dataset, the CPU cache miss rate is 36.48%, which is smaller than that of Flood, LISA and IFI. However, as indicated by the green cell colors, the range query processing time for Flood, LISA and IFI are all less than that of ZMI. The reason is that the above three learned indexes have a better capability of pruning unnecessary candidates given a specific query range, leading to less memory footprint (i.e., small total cache references count). As an example, the count of total cache references (e.g., 50M, 46M, and 47M for IFI, LISA and Flood, respectively) are all smaller than the 73M references produced by ZMI. As a consequence, the overall effect of smaller total cache references results in fewer query processing costs.

Takeaways. There is a general positive correlation between the query processing time and the CPU performance counters (CR, CMR, BR, BMR). Though demonstrated to be memory-bound, performant learned indexes, such as Flood, LISA, and IFI, usually have better pruning capabilities, resulting in fewer cache references, and eventually leading to faster query processisng.

5.10 Summary of results

Based on the results, we summarize our evaluation of each learned index in Table 8, focusing on two key aspects: index construction and query processing.

Index Construction (Q1, Q2, Q6). Regarding the cost of index construction (see the Construction Time column in Table 8), learned indexes that utilize PGMIndex as their internal model are generally as fast as non-learned baselines. In contrast, deep learning-based learned indexes tend to be significantly slower than non-learned baselines, primarily due to the substantial runtimes associated with deep learning.

As for the memory footprint of the constructed index (see the Memory Cost column in Table 8), multi-dimensional learned indexes typically demonstrate robust and significant space reduction compared to non-learned indexes, such as R-tree.

Additionally, our evaluation results indicate that, unlike in the one-dimensional scenario, the error parameter ϵ for multi-dimensional learned indexes has minimal impact on index construction time and and query efficiency. Therefore, by carefully balancing efficiency and space cost, we recommend setting ϵ to **64** to achieve satisfactory performance.

Query Processing (Q3, Q4, Q5, Q7, Q8). Regarding range query processing efficiency (refer to the corresponding column in Table 8), learned indexes such as IFI, Flood, and LISA consistently outperform non-learned baselines like R-tree variants across queries with varying selectivity levels. The speedup ratio is particularly notable for datasets with a uniform distribution and for queries with low selectivity.

The evaluation results of the kNN query processing (see the kNN Query Efficiency column in Table 8) contrast with our findings for range queries, where no learned index consistently outperform non-learned baselines such as kd-tree variants. The progressive nature of range searches limits the efficiency of kNN query processing in learned indexes.

As for the scalability (see the Scalability column in Table 8), most methods scale well with large datasets (up to $N = 100$M) and moderate dimensions (up to $d = 8$), with an exception of RMSI. We were unable to train the underlying deep learning models for RMSI when the data size reaches 50M.

In terms of handling dynamic operations (see the Dynamic Update column in Table 8), existing learned index structures either lack support for dynamic operations or offer only limited support. As a result, they do not consistently outperform conventional indexes such as R-tree and kd-trees.

Regarding the support for extended geometries such as bounding boxes, we found that efficient indexes like Flood and IFI also perform well with box queries. There is a strong linear correlation between processing time for box and point queries, indicated by a coefficient of determination of $R^2 = 0.9824$.

Performance Analysis (Q9). Our deep-dive analysis into the performance of learned indexes in Sect. 5.9 reveals that, there is a general positive correlation between the query processing time and hardware performance counter statistics (e.g., cache reference counts and cache miss rates). The most performant learned indexes such as Flood, LISA and IFI generally have a better capability of pruning unnecessary candidates given a specific range query, resulting in fewer cache references, and a better capability of utilizing cache locality in their data layout design. In addition to cache ref-

Table 8 Summary of evaluation results. The cell colors from left to right ■ ■ ■ ■ represent the performance levels of each evaluation item from the worst to the best

Index	Index Construction Evaluation			Query Processing Evaluation			
	Construction Time	Memory Cost	Parameter Tunnig	Range Query Efficiency	kNN Query Efficiency	Scalability	Dynamic Update
ZMI	better than R-tree	excellent	easy	sometimes better than R-tree	sometimes better than kd-tree	excellent	N.A.
HMI	better than R-tree	excellent	easy	sometimes better than R-tree	N.A.	excellent	N.A.
SLBRIN	better than R-tree	excellent	easy	sometimes better than R-tree	N.A.	excellent	N.A.
Wazi	comparable to R-tree	excellent	easy	sometimes better than R-tree	N.A.	excellent	N.A.
LMSFC	comparable to R-tree	excellent	easy	sometimes better than R-tree	N.A.	excellent	N.A.
MLI	comparable to R-tree	excellent	easy	no better than R-tree	sometimes better than kd-tree	good	N.A.
IFI	comparable to R-tree	good	N.A.	robustly better than R-tree	N.A.	excellent	limited support
RSMI	much larger than R-tree	good	N.A.	sometimes better than R-tree	ANN is better than kd-tree	bad	limited support
LISA	comparable to R-tree	good	easy	robustly better than R-tree	sometimes better than kd-tree	excellent	limited support
Flood	comparable to R-tree	good	easy	robustly better than R-tree	N.A.	excellent	N.A.

erences, methods with a higher number of CPU branches or higher branch miss rates tend to have longer query processing time.

6 Conclusion and future studies

6.1 Conclusion

Efficient indexing holds paramount importance for multi-dimensional data management and analytics. Inspired by the seminal work on 1-D learned indexes [44], there is a growing trend of utilizing machine learning models to directly learn storage layouts for multi-dimensional data. In this work, we provide a comprehensive experimental study to evaluate 10 representative indexes under a uniform configuration, aiming to answer the core question: *How good are multi-dimensional learned indexes?* The key experimental findings are three-fold.

F1: Compared to traditional indexes, multi-dimensional learned indexes (especially `LISA`, `Flood`, and `IFI`) can significantly reduce the index size (up to 3 orders of magnitude) while achieving better **range** query processing efficiency (up to 2.19×).

F2: In terms of kNN query processing and dynamic operations, learned indexes **cannot** consistently outperform conventional methods due to the lack of proper query processing algorithms and the intrinsic hardness of updating models and data layouts at the same time.

F3: We identify the correlations between the performance of learned indexes and hardware performance counters, revealing that efficient learned indexes are usually cache-efficient, typically exhibiting stronger candidate pruning power (reflected in fewer cache references) and better data layouts that maintain spatial locality (indicated by lower cache miss rates).

6.2 Future studies

Learned index design on multi-dimensional data is a prominent and rapidly growing field in the data management community. Our comprehensive experimental study reveals that, though achieving significant improvements in **some** cases, multiple technical challenges remain to be addressed before multi-dimensional learned indexes can be widely applied in practical systems. Based on our key experimental findings, we identify the following potential research opportunities.

Advanced Query Support. According to **F1** and **F2**, existing multi-dimensional learned indexes excel in accelerating **range** query processing while significantly reducing index space costs. However, support for kNN queries and more advanced analytical operators OLAP workloads, such as skyline queries [13] and spatial joins [36, 85], is either limited or missing. Extending existing learned index structures or designing new learned data layouts to efficiently support these analytical queries presents an interesting and challenging opportunity.

Efficient Updatable Index. As discussed in Sect. 3.6 and noted in **F2**, most existing learned indexes focus on the **read-only** workloads. Although indexes like `RSMI` [69] and `LISA` [47] support dynamic operations through a model-based data layout, re-training the entire model becomes necessary when the data distribution shifts significantly from the one used to

build the index. Developing a fully updatable learned multi-dimensional index will require novel data layout designs and efficient model update strategies. Additionally, a well-crafted cost model should be established to seamlessly embed learned indexes into DBMS with cost-based optimizers.

IO-Efficient and Cache-Efficient Learned Index. Existing works on learned indexes mainly target on **in-memory** query processing. Although `LISA` [47] and `RSMI` [69] claim that they can be adapted for disk-based storage, they do not specifically optimize for disk access characteristics. In addition, although **F3** claims that some learned index structures are cache-efficient, there are still optimization opportunities to further reduce the cache miss rate by designing new ML-based data layouts that carefully consider the memory hierarchy.

Distributed Spatial Analytics. To process and analyze web-scale multi-dimensional data, practical solutions usually adopt distributed computation engines like Spark [8] to serve as the back-end, e.g., LocationSpark [76], GeoSpark [90], Simba [86]. As reported in an evaluation study [65], the index memory overhead of existing systems is generally high. Given the significant space-time trade-off noted in **F1**, designing new distributed multi-dimensional analytical systems powered by learned indexes is a promising direction.

Theoretical Performance Modeling. Theoretically answering the question of why learned indexes are effective is a prominent open problem. Recent studies on 1D learned indexes [24, 49, 91, 92] claim that, on N sorted keys, learned indexes can process lookup queries in sub-logarithmic time $O(\log \log N)$ while using linear space $O(N)$, demonstrating provable better performance compared to conventional B+-tree indexes. However, similar theoretical results are currently lacking for multi-dimensional learned indexes, making it an interesting research problem to extend these results from 1-dimensional to multi-dimensional scenarios.

Acknowledgements Qiyu Liu is supported by fundamental research funds for the central universities of the Ministry of Education of China (No. 5330501211). Yuxiang Zeng's work is partially supported by Key Research and Development Program of China under Grant No. 2023YFF0725103, National Science Foundation of China (NSFC) (Grant Nos. U21A20516, 62425202, 62336003), and Beijing Natural Science Foundation (Z230001). Yanyan Shen is supported by the National Key Research and Development Program of China (2022YFE0200500) and Shanghai Municipal Science and Technology Major Project (2021SHZDZX0102). Lei Chen's work is partially supported by National Key Research and Development Program of China Grant No. 2023YFF0725100, National Science Foundation of China (NSFC) under Grant No. U22B2060, the Hong Kong RGC GRF Project 16213620, RIF Project R6020-19, AOE Project AoE/E-603/18, Theme-based project TRS T41-603/20R, CRF Project C2004-21G, Guangdong Province Science and Technology Plan Project 2023A0505030011, Guangzhou municipality big data intelligence key lab, 2023A03J0012, Hong Kong ITC ITF grants MHX/078/21 and PRP/004/22FX, Zhujiang scholar program 2021JC02X170, Microsoft Research Asia Collaborative Research Grant, HKUST-Webank joint research lab and 2023 HKUST Shenzhen-Hong Kong Collaborative Innovation Institute Green Sustainability Special Fund, from Shui On Xintiandi and the InnoSpace GBA.

References

1. Aggarwal, C.C., Hinneburg, A., Keim, D.A.: On the surprising behavior of distance metrics in high dimensional space. In: ICDT 2001, Proceedings 8, pp. 420–434. Springer (2001)
2. Andoni, A., Indyk, P.: Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In: FOCS, pp. 459–468. IEEE Computer Society (2006)
3. ANN Project: http://www.cs.umd.edu/~mount/ANN/. Accessed: 2024-04-15
4. Arge, L., de Berg, M., Haverkort, H.J., Yi, K.: The priority r-tree: a practically efficient and worst-case optimal r-tree. ACM Trans. Algorithms **4**(1), 9:1–9:30 (2008)
5. Arthur, D., Vassilvitskii, S.: k-means++: the advantages of careful seeding. In: SODA, pp. 1027–1035. SIAM (2007)
6. Arya, S., Mount, D.M., Netanyahu, N.S., Silverman, R., Wu, A.Y.: An optimal algorithm for approximate nearest neighbor searching fixed dimensions. J. ACM (JACM) **45**(6), 891–923 (1998)
7. Aumüller, M., Bernhardsson, E., Faithfull, A.: Ann-benchmarks: a benchmarking tool for approximate nearest neighbor algorithms. Inf. Syst. **87**, 101374 (2020)
8. Apache Spark. https://spark.apache.org/. Accessed: 2024-04-15
9. Beckmann, N., Kriegel, H., Schneider, R., Seeger, B.: The R*-tree: an efficient and robust access method for points and rectangles. In: SIGMOD Conference, pp. 322–331. ACM Press (1990)
10. Bentley, J.L.: Multidimensional binary search trees used for associative searching. Commun. ACM **18**(9), 509–517 (1975)
11. Boffa, A., Ferragina, P., Vinciguerra, G.: A "learned" approach to quicken and compress rank/select dictionaries. In: ALENEX, pp. 46–59. SIAM (2021)
12. Boost Geometry. http://boost.org/libs/geometry
13. Börzsönyi, S., Kossmann, D., Stocker, K.: The skyline operator. In: ICDE, pp. 421–430. IEEE Computer Society (2001)
14. Bozkaya, T., Ozsoyoglu, M.: Indexing large metric spaces for similarity search queries. ACM Trans. Database Syst. (TODS) **24**(3), 361–404 (1999)
15. Chen, H., Chiang, R.H., Storey, V.C.: Business intelligence and analytics: from big data to big impact. MIS Q. 1165–1188 (2012)
16. Chen, L., Gao, Y., Zheng, B., Jensen, C.S., Yang, H., Yang, K.: Pivot-based metric indexing. Proc. VLDB Endow. **10**(10) (2017)
17. Chen, Q., Li, D., Tang, C.K.: Knn matting. IEEE Trans. Pattern Anal. Mach. Intell. **35**(9), 2175–2188 (2013)
18. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.S.: Locality-sensitive hashing scheme based on p-stable distributions. In: SCG, pp. 253–262. ACM (2004)

19. Davitkova, A., Milchevski, E., Michel, S.: The ML-index: a multi-dimensional, learned index for point, range, and nearest-neighbor queries. In: EDBT, pp. 407–410. OpenProceedings.org (2020)
20. Ding, J., Minhas, U.F., Yu, J., Wang, C., Do, J., Li, Y., Zhang, H., Chandramouli, B., Gehrke, J., Kossmann, D., Lomet, D.B., Kraska, T.: ALEX: an updatable adaptive learned index. In: SIGMOD Conference, pp. 969–984. ACM (2020)
21. Ding, J., Nathan, V., Alizadeh, M., Kraska, T.: Tsunami: a learned multi-dimensional index for correlated data and skewed workloads. Proc. VLDB Endow. **14**(2), 74–86 (2020)
22. Daily Historical Stock Prices. https://www.kaggle.com/ehallmar/daily-historical-stock-prices-1970-2018. Accessed: 2024-10-11
23. Faghmous, J.H., Kumar, V.: Spatio-temporal data mining for climate data: advances, challenges, and opportunities. In: Data mining and knowledge discovery for big data, pp. 83–116. Springer (2014)
24. Ferragina, P., Lillo, F., Vinciguerra, G.: Why are learned indexes so effective? In: ICML. Proceedings of Machine Learning Research, vol. 119, pp. 3123–3132. PMLR (2020)
25. Ferragina, P., Vinciguerra, G.: The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. Proc. VLDB Endow. **13**(8), 1162–1175 (2020)
26. Finkel, R.A., Bentley, J.L.: Quad trees a data structure for retrieval on composite keys. Acta Inf. **4**, 1–9 (1974)
27. FourSquare Data. https://sites.google.com/site/yangdingqi/home/foursquare-dataset. Accessed: 2024-04-15
28. Gao, J., Cao, X., Yao, X., Zhang, G., Wang, W.: LMSFC: a novel multidimensional index based on learned monotonic space filling curves. Proc. VLDB Endow. **16**(10), 2605–2617 (2023)
29. Ge, T., He, K., Ke, Q., Sun, J.: Optimized product quantization. IEEE Trans. Pattern Anal. Mach. Intell. **36**(4), 744–755 (2014)
30. GEOS. https://github.com/libgeos/geos. Accessed: 2024-04-15
31. gperftools: https://github.com/gperftools/gperftools. Accessed: 2024-04-15
32. Gu, T., Feng, K., Cong, G., Long, C., Wang, Z., Wang, S.: The RLR-tree: a reinforcement learning based r-tree for spatial data. arXiV:2103.04541 (2021)
33. Guttman, A.: R-trees: A dynamic index structure for spatial searching. In: SIGMOD Conference, pp. 47–57. ACM Press (1984)
34. Hadian, A., Kumar, A., Heinis, T.: Hands-off model integration in spatial index structures. In: AIDB@VLDB (2020)
35. Haider, C.M.R., Wang, J., Aref, W.G., et al.: The "ai+ r"-tree: An instance-optimized r-tree. In: 2022 23rd IEEE International Conference on Mobile Data Management (MDM), pp. 9–18. IEEE (2022)
36. Hjaltason, G.R., Samet, H.: Incremental distance join algorithms for spatial databases. In: SIGMOD Conference, pp. 237–248. ACM Press (1998)
37. Huber, P.J.: Robust estimation of a location parameter. In: Breakthroughs in Statistics: Methodology and Distribution, pp. 492–518. Springer (1992)
38. Jagadish, H.V., Ooi, B.C., Tan, K., Yu, C., Zhang, R.: iDistance: an adaptive B^+-tree based indexing method for nearest neighbor search. ACM Trans. Database Syst. **30**(2), 364–397 (2005)
39. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011)
40. Kamel, I., Faloutsos, C.: Hilbert R-tree: an improved R-tree using fractals. In: VLDB, pp. 500–509. Morgan Kaufmann (1994)
41. Kanth, K.V.R., Ravada, S., Abugov, D.: Quadtree and r-tree indexes in oracle spatial: a comparison using GIS data. In: SIGMOD Conference, pp. 546–557. ACM (2002)
42. Kipf, A., Marcus, R., van Renen, A., Stoian, M., Kemper, A., Kraska, T., Neumann, T.: Radixspline: a single-pass learned index. In: aiDM@SIGMOD, pp. 5:1–5:5. ACM (2020)
43. Kraska, T., Alizadeh, M., Beutel, A., Chi, E.H., Kristo, A., Leclerc, G., Madden, S., Mao, H., Nathan, V.: SageDB: a learned database system. In: CIDR. www.cidrdb.org (2019)
44. Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N.: The case for learned index structures. In: SIGMOD Conference, pp. 489–504. ACM (2018)
45. Kristo, A., Vaidya, K., Çetintemel, U., Misra, S., Kraska, T.: The case for a learned sorting algorithm. In: SIGMOD Conference, pp. 1001–1016. ACM (2020)
46. Leutenegger, S.T., Edgington, J.M., López, M.A.: STR: A simple and efficient algorithm for R-tree packing. In: ICDE, pp. 497–506. IEEE Computer Society (1997)
47. Li, P., Lu, H., Zheng, Q., Yang, L., Pan, G.: LISA: A learned index structure for spatial data. In: SIGMOD Conference, pp. 2119–2133. ACM (2020)
48. Liu, H., Wang, R., Shan, S., Chen, X.: Deep supervised hashing for fast image retrieval. In: CVPR, pp. 2064–2072. IEEE Computer Society (2016)
49. Liu, Q., Han, S., Qi, Y., Peng, J., Li, J., Lin, L., Chen, L.: Why are learned indexes so effective but sometimes ineffective? arXiv preprint arXiv:2410.00846 (2024)
50. Liu, Q., Shen, Y., Chen, L.: Hap: an efficient hamming space index based on augmented pigeonhole principle. In: Proceedings of the 2022 International Conference on Management of Data, pp. 917–930 (2022)
51. Liu, Q., Zheng, L., Shen, Y., Chen, L.: Stable learned bloom filters for data streams. Proc. VLDB Endow. **13**(11), 2355–2367 (2020)
52. Malkov, Y., Ponomarenko, A., Logvinov, A., Krylov, V.: Approximate nearest neighbor algorithm based on navigable small world graphs. Inf. Syst. **45**, 61–68 (2014)
53. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4), 824–836 (2020)
54. Marcus, R., Kipf, A., van Renen, A., Stoian, M., Misra, S., Kemper, A., Neumann, T., Kraska, T.: Benchmarking learned indexes. Proc. VLDB Endow. **14**(1), 1–13 (2020)
55. Mitzenmacher, M.: A model for learned bloom filters and optimizing by sandwiching. In: NeurIPS, pp. 462–471 (2018)
56. morton-nd: https://github.com/morton-nd/morton-nd. Accessed: 2024-04-15
57. nanoflann: https://github.com/jlblancoc/nanoflann
58. Nathan, V., Ding, J., Alizadeh, M., Kraska, T.: Learning multi-dimensional indexes. In: SIGMOD Conference, pp. 985–1000. ACM (2020)
59. Nievergelt, J., Hinterberger, H., Sevcik, K.C.: The grid file: an adaptable, symmetric multikey file structure. ACM Trans. Database Syst. **9**(1), 38–71 (1984)
60. Numpy: https://numpy.org/
61. NYC Yellow Taxi Trip Data: https://www.kaggle.com/datasets/elemento/nyc-yellow-taxi-trip-data. Accessed: 2024-10-11
62. O'Rourke, J.: An on-line algorithm for fitting straight lines between data ranges. Commun. ACM **24**(9), 574–578 (1981)
63. OpenStreet Map: https://planet.openstreetmap.org. Accessed: 2024-04-15
64. Pai, S., Mathioudakis, M., Wang, Y.: Wazi: A learned and workload-aware z-index. In: EDBT, pp. 559–571 (2024)
65. Pandey, V., Kipf, A., Neumann, T., Kemper, A.: How good are modern spatial analytics systems? Proc. VLDB Endow. **11**(11), 1661–1673 (2018)
66. Pandey, V., van Renen, A., Kipf, A., Kemper, A.: How good are modern spatial libraries? Data Sci. Eng. **6**(2), 192–208 (2021)
67. PostgreSQL: Postgresql: The world's most advanced open source relational database. Web resource: https://www.postgresql.org/ (2021)
68. PyTorch: https://pytorch.org/

69. Qi, J., Liu, G., Jensen, C.S., Kulik, L.: Effectively learning spatial indices. Proc. VLDB Endow. **13**(11), 2341–2354 (2020)
70. Qi, J., Tao, Y., Chang, Y., Zhang, R.: Packing R-trees with space-filling curves: theoretical optimality, empirical efficiency, and bulk-loading parallelizability. ACM Trans. Database Syst. **45**(3), 14:1–14:47 (2020)
71. Ramsak, F., Markl, V., Fenk, R., Zirkel, M., Elhardt, K., Bayer, R.: Integrating the UB-tree into a database system kernel. In: VLDB, pp. 263–272. Morgan Kaufmann (2000)
72. Robinson, J.T.: The kdb-tree: a search structure for large multi-dimensional dynamic indexes. In: Proceedings of the 1981 ACM SIGMOD international conference on Management of data, pp. 10–18 (1981)
73. Rosenfeld, V., Breß, S., Markl, V.: Query processing on heterogeneous CPU/GPU systems. ACM Comput. Surv. (CSUR) **55**(1), 1–38 (2022)
74. Sejnowski, T.J., Churchland, P.S., Movshon, J.A.: Putting big data to good use in neuroscience. Nat. Neurosci. **17**(11), 1440–1441 (2014)
75. Sun, Z., Zhou, X., Li, G.: Learned index: A comprehensive experimental evaluation. Proc. VLDB Endow. **16**(8), 1992–2004 (2023)
76. Tang, M., Yu, Y., Malluhi, Q.M., Ouzzani, M., Aref, W.G.: LocationSpark: a distributed in-memory data management system for big spatial data. Proc. VLDB Endow. **9**(13), 1565–1568 (2016)
77. Tao, Y., Papadias, D., Sun, J.: The TPR*-tree: an optimized spatio-temporal access method for predictive queries. In: VLDB, pp. 790–801. Morgan Kaufmann (2003)
78. Tensorflow: https://www.tensorflow.org/
79. Toronto3D Data: https://github.com/WeikaiTan/Toronto-3D. Accessed: 2024-04-15
80. TPC-H Homepage: http://www.tpc.org/tpch/. Accessed: 2024-10-11
81. Wang, H., Fu, X., Xu, J., Lu, H.: Learned index for spatial queries. In: MDM, pp. 569–574. IEEE (2019)
82. Wang, L., Hu, L., Fu, C., Yu, Y., Tang, P., Zhang, F., Liu, R.: SLBRIN: a spatial learned index based on brin. ISPRS Int. J. Geo Inf. **12**(4), 171 (2023)
83. Wang, N., Xu, J.: Spatial queries based on learned index. In: Spatial Data and Intelligence: First International Conference, SpatialDI 2020, Virtual Event, May 8–9, 2020, Proceedings 1, pp. 245–257. Springer (2021)
84. Wu, J., Zhang, Y., Chen, S., Chen, Y., Wang, J., Xing, C.: Updatable learned index with precise positions. Proc. VLDB Endow. **14**(8), 1276–1288 (2021)
85. Xia, C., Lu, H., Ooi, B.C., Hu, J.: Gorder: An efficient method for KNN join processing. In: VLDB, pp. 756–767. Morgan Kaufmann (2004)
86. Xie, D., Li, F., Yao, B., Li, G., Zhou, L., Guo, M.: Simba: Efficient in-memory spatial analytics. In: SIGMOD Conference, pp. 1071–1085. ACM (2016)
87. Xu, D., Tsang, I.W., Zhang, Y.: Online product quantization. IEEE Trans. Knowl. Data Eng. **30**(11), 2185–2198 (2018)
88. Yang, Z., Chandramouli, B., Wang, C., Gehrke, J., Li, Y., Minhas, U.F., Larson, P., Kossmann, D., Acharya, R.: Qd-tree: Learning data layouts for big data analytics. In: SIGMOD Conference, pp. 193–208. ACM (2020)
89. Yianilos, P.N.: Data structures and algorithms for nearest neighbor. In: Proceedings of the fourth annual ACM-SIAM Symposium on Discrete algorithms, vol. 66, p. 311. SIAM (1993)
90. Yu, J., Wu, J., Sarwat, M.: GeoSpark: a cluster computing framework for processing large-scale spatial data. In: SIGSPATIAL/GIS, pp. 70:1–70:4. ACM (2015)
91. Zeighami, S., Shahabi, C.: Theoretical analysis of learned database operations under distribution shift through distribution learnability. In: 41st ICML
92. Zeighami, S., Shahabi, C.: Towards establishing guaranteed error for learned database operations. In: The 12th ICLR
93. Zhu, L., Yu, F.R., Wang, Y., Ning, B., Tang, T.: Big data analytics in intelligent transportation systems: A survey. IEEE Trans. Intell. Transp. Syst. **20**(1), 383–398 (2019)